GPU-First Heisenberg-Picture Tensor Network Dynamics for the 2D Transverse-Field Ising Model

Paolo D'Alberto*
Advanced Micro Devices, Inc.
paolo.dalberto@amd.com

May 28, 2026

Abstract

We present CppSim, a C++/HIP 2D Ising simulator for Heisenberg-picture tensor network time evolution on AMD GPUs. The key computational contributions are: first, a zero-malloc GPU workspace that pre-allocates all buffers at startup; second, a custom HIP tensor permutation kernel replacing host-side index shuffling with a pure device-to-device operation, yielding a 7.6× trotter speedup; third, a hybrid QR strategy selecting Cholesky-QR for tall-skinny matrices and Householder-QR otherwise; fourth, adaptive Belief Propagation with log-space Bethe partition function evaluation and explicit sign tracking. The key validation contributions are: first, correctness and demonstration of χ-convergence of the spin autocorrelation function $C(t)$ for $\chi \in \{20, \ldots, 110\}$ on a 4×4 grid, identifying $\chi^* \approx 60$ as the practical convergence point. Second, we characterize BP fixed-point multiplicity and introduce a four-step trap remediation procedure using damped BP updates. Third, we identify operator-placement C_4 symmetry as the root cause of BP fixed-point multiplicity — symmetric grids are entirely trap-free. Fourth, grid scaling from 3×3 to 10×10 demonstrates operator lightcone saturation consistent with the Lieb–Robinson bound. Cross-architecture comparison between MI100 and MI60 reveals complementary trap patterns and defines portable operating points at $\chi \in \{40, 60\}$. An analytic memory model, validated to within 2% against hardware measurements, guided deployment on AMD MI355X (309 GB HBM3E), where we demonstrate the first tensor-network simulation of the 2D Ising model on a 10×10 lattice at $\chi_{\max} = 100$ (≈ 109 GB) and an 11×11 lattice at $\chi_{\max} = 100$ (≈ 130 GB). The 11×11 C4-symmetric run is entirely trap-free and reveals a clean three-phase operator spreading: ballistic Lieb–Robinson cone, boundary reflection ring, and onset of full scrambling.

1 Introduction

Simulating real-time quantum dynamics of strongly correlated many-body systems is a central challenge in computational physics. Tensor network (TN) methods provide a systematically improvable approximation by representing quantum operators as networks of low-rank tensors [1], with accuracy controlled by the bond dimension χ. While one-dimensional systems are well-served by matrix product state (MPS) methods [2], two-dimensional (2D) tensor networks are increasingly important for studying frustrated magnets, quantum circuits, and non-equilibrium dynamics at accessible system sizes.

The broader motivation for this work is the multi-product formula (MPF) framework for hybrid

*Advanced Micro Devices, Inc. (AMD). paolo.dalberto@amd.com. This work was developed in active collaboration with Claude (Anthropic)

quantum-classical computation. In MPF, a quantum computer executes Trotter circuits to approximate Hamiltonian time evolution, and a classical linear regression over multiple circuit runs at different step sizes cancels the leading Trotter error terms. The spin autocorrelation function $C(t) = \mathrm{Tr}[Z_c(t)\, Z_c]/2^N$ is the quantitative figure of merit: it is what the ideal, error-free circuit would compute, and what the MPF post-processing corrects toward. CppSim serves as a simulation sandbox: by computing the ideal $C(t)$ alongside the Trotter-approximate trajectory, it enables direct measurement of the circuit error and systematic exploration of MPF compensation strategies before deployment on quantum hardware. Throughout this work, all simulations are *noise-free* — the only approximation is bond-dimension truncation; extension to noisy gates and alternative lattice geometries (honeycomb, heavy-hex) is left to future work.

The *Heisenberg picture* [9] is the natural language for this setting: the observable Z_c is evolved forward in time under the Hamiltonian while the infinite-temperature state remains fixed, which maps directly to how a quantum circuit acts on an operator. The correlation function is then extracted as an inner product with the initial operator — no state preparation or measurement of a full density matrix is required. Computing $C(t)$ exactly would require contracting the full 2D tensor network, an exponentially costly operation. Belief propagation (BP) [10] makes this tractable: each lattice site iteratively sends its neighbors a message summarizing everything it sees from its side of the network; after convergence, each site holds an approximate picture of the global environment using only local information. The approximation rests on the Bethe assumption: on a tree, a site's dependence on the rest of the network passes entirely through its direct neighbors — there are no alternative paths — making BP exact.

Existing implementations of these methods are primarily in Julia [9], benefiting from high-level tensor abstractions and just-in-time compilation. However, Julia's garbage collector imposes nontrivial overhead for GPU workloads: allocation patterns during simulation trigger frequent collection cycles, and GPU memory pool management introduces latency spikes. For production-scale computations at large bond dimensions and grid sizes, these overheads become significant.

We present CppSim, a C++/HIP implementation that addresses these limitations through explicit GPU memory management, GPU-native linear algebra, and a zero-malloc simulation loop, eliminating the memory pool and its management overhead entirely. The implementation targets AMD GPUs through the HIP portability layer and the rocBLAS/rocSOLVER libraries [4, 5].

The main computational contributions of this work are:

1. A **zero-malloc GPU workspace** design that pre-allocates all buffers at startup, making peak VRAM usage known exactly before the first layer runs and eliminating all allocator overhead during simulation.
2. A **custom HIP tensor permutation kernel** that replaces host-side index shuffling (requiring D2H+H2D transfers) with a pure device-to-device operation, yielding a 7.6× trotter speedup.
3. An **adaptive QR strategy** combining Cholesky-QR for tall-thin matrices at saturation with Householder-QR for small bond dimensions, with explicit numerical safeguards.
4. A **sign-correct log-space Bethe partition function** with explicit Bethe edge correction and per-bond sign tracking, resolving systematic C(t) anomalies at late times.

Because all buffer sizes are determined analytically before the first layer runs, the layout preallocates exactly the peak working set at $\chi_{\max}$: the minimum memory consistent with full parallel

execution at saturation, where every intermediate tensor — contracted site tensors, QR factors, SVD matrices, environment buffers, and solver scratch — occupies a named, fixed slot and no dynamic allocation occurs during the simulation loop. This allocation is *asymptotically optimal*: the four large buffers (two contracted site tensors and two QR orthogonal factors, each of size $O(\chi_{\max}^4)$) must coexist during every gate application and therefore set an unavoidable lower bound; all remaining buffers are $O(\chi_{\max}^2)$ or smaller and do not contribute to the leading term. Six of the nine named buffers are reused across pipeline stages within a single gate, reducing the constant prefactor without sacrificing the self-documenting, named-slot layout. The resulting closed-form model (`estimate_memory.py`) predicts VRAM requirements for any grid and bond dimension to within 2% of hardware measurements, enabling deployment decisions across GPU generations before a single simulation step is taken.

The remainder of this paper is organized as follows. Section 2 describes the mathematical formulation. Section 3 presents the software design. Section 4 details the core algorithms. Section 5 describes the implementation. Section 6 presents numerical experiments and physics results. Section 7 reports performance. Section 8 discusses GPU portability. Section 9 concludes.

2 Mathematical Formulation

This section establishes the physical model and the mathematical framework underlying the simulation. We work entirely in the Heisenberg picture, where the observable rather than the quantum state is evolved in time, and we represent it as a tensor network on the lattice graph.

2.1 Model

We consider the 2D transverse-field Ising model [6] on a $k \times l$ rectangular lattice with open boundary conditions and Hamiltonian:

$$\mathcal{H} = -J \sum_{\langle ij \rangle} X_i X_j \; - \; h \sum_i Z_i, \tag{1}$$

where X_i, Z_i are Pauli operators on site i, J is the Ising coupling, h is the transverse field strength, and the sum is over nearest-neighbor pairs.

2.2 Heisenberg Picture and Spin Autocorrelation

The spin autocorrelation function measures how much the observable Z_c at the lattice center remembers its initial value after time t:

$$C(t) = \mathrm{Tr}\big[\rho_0 \, Z_c(t) \, Z_c\big], \tag{2}$$

where ρ_0 is the initial state. In the Heisenberg picture, the state ρ_0 remains fixed while the observable evolves under the Hamiltonian:

$$Z_c(t) = e^{i\mathcal{H}t} \, Z_c \, e^{-i\mathcal{H}t}. \tag{3}$$

Choosing ρ_0 constant (proportional to the identity), the trace simplifies to:

$$C(t) = \frac{1}{2^N} \mathrm{Tr}\big[Z_c(t) \, Z_c\big], \tag{4}$$

where 2^N is the Hilbert space dimension (N sites), which is the quantity computed throughout this work and targeted by the MPF post-processing described in Section 1.

2.3 Pauli Basis Tensor Network

Each qubit site has a 2-dimensional Hilbert space, but the space of 2×2 Hermitian operators has dimension $D = 4$, spanned by the Pauli matrices $\{I, X, Y, Z\}$ — orthogonal under the Hilbert-Schmidt inner product $\mathrm{Tr}(A^\dagger B)$. The operator $Z_c(t)$ is represented as a tensor network on the lattice graph: each site v carries a tensor ψ_v whose indices encode both the bond entanglement with neighboring sites and the local Pauli component of the operator.

Concretely, $\deg(v)$ denotes the number of nearest-neighbor bonds of site v: 2 at corners, 3 at edge sites, and 4 at interior sites on a rectangular grid. The tensor ψ_v is pre-allocated with one bond index of dimension $\chi_{\max}$ per neighbor and one Pauli index of dimension $D = 4$, giving a maximum shape of $\chi_{\max}^{\deg(v)} \times D$. The live bond dimension χ_{live} grows from 1 to $\chi_{\max}$ as entanglement builds across the first Trotter layers; all computations use the current live dimension, not the pre-allocated maximum. The entry $\psi_v[i_1, \ldots, i_{\deg(v)}, p]$ is the coefficient of the p-th Pauli matrix (p=0: I, p=1: X, p=2: Y, p=3: Z) at site v, for bond state $(i_1, \ldots, i_{\deg(v)})$. At $t = 0$: the center site has $\psi_c[\cdot, 3] = 1$ (pure Z); all others have $\psi_v[\cdot, 0] = 1$ (pure I).

The bond entanglement between neighboring sites a and b is captured by one $\chi \times \chi$ message matrix $\mu_{a\to b}$ per directed bond. During Belief Propagation, each $\mu_{a\to b}$ is a general positive-semidefinite matrix updated iteratively until a self-consistent fixed point is reached. At that fixed point the matrices become diagonal, and their diagonal entries are the *Schmidt singular values* of the bipartition of the network across bond (a, b): each entry $\lambda_k \geq 0$ is the weight of one entanglement channel across the cut, in the same way that the singular values of a matrix decompose it into rank-one pieces ordered by importance. A product state (no entanglement) has only $\lambda_1 > 0$; a maximally entangled bond has all χ values equal. This fixed point is the *Vidal canonical gauge* [3], in which the site tensors are orthogonalized so that all entanglement information is encoded in the diagonal messages, and truncating each bond to rank $\chi_{\max}$ discards only the least significant channels — the locally optimal low-rank approximation at fixed bond dimension. On a tree this gauge is exact; on the 2D lattice BP finds an approximate Vidal gauge, which is why BP runs before every correlation function evaluation.

2.4 Trotter Decomposition

Each time step δt advances the operator by unitary conjugation:

$$Z_c(t + \delta t) = U^\dagger(\delta t)\; Z_c(t)\; U(\delta t), \qquad U(\delta t) = e^{-i\mathcal{H}\delta t}, \tag{5}$$

where $U^\dagger = e^{+i\mathcal{H}\delta t}$ acts on the left and U on the right. Because the Hamiltonian (1) contains only real Pauli couplings, this conjugation maps Pauli matrices to real linear combinations of Pauli matrices, so the gate acting on ψ_v in the Pauli basis is a real orthogonal matrix — no complex arithmetic is required.

The unitary $U(\delta t)$ is itself approximated to first order as a composition of simpler gates (the Trotter decomposition [7]):

$$U(\delta t) \approx R_z(h\,\delta t) \cdot R_{xx}(2J\,\delta t) \cdot R_z(h\,\delta t), \tag{6}$$

where R_z is a single-site rotation (diagonal in the Pauli basis) and R_{xx} is a two-site entangling gate applied to all nearest-neighbor pairs via a 4-color decomposition: bonds are partitioned into four color classes such that no two bonds in the same class share a site, enabling sequential independent updates.

2.5 Gate Update

For each two-site gate on bond (v_1, v_2), let $i \in \{1, 2\}$ index the two sites. The message matrices $\mu_{a\to b}$ (defined in §2) are diagonal in the Vidal gauge, holding singular values; for the bond under update, $\sqrt{\mu_{v_1\to v_2}}$ and $\sqrt{\mu_{v_2\to v_1}}$ denote the diagonal matrices of their square roots, and $\sqrt{\mu_{e\to v_i}}$ denotes the same for every other incident bond e. The update proceeds as follows (see Algorithm 1):

1. **Environment absorption.** Contract ψ_{v_i} with $\sqrt{\mu_{e\to v_i}}$ on every bond e incident to v_i except the active bond (v_1, v_2). This yields the *contraction vector* $\mathbf{cv}_i$ — so named because it is the site tensor contracted with its environment, ready for QR — which folds the environmental context into each site before the update.
2. **QR decomposition.** Factor each gauged tensor: $\mathbf{cv}_i = \mathbf{Q}_i\mathbf{R}_i$, where $\mathbf{Q}_i$ ($\chi_{\max}^3 \times D\chi_{\max}$) is isometric and $\mathbf{R}_i$ ($D\chi_{\max} \times D\chi_{\max}$) is compact. QR reduces the subsequent SVD from the intractable full tensor ($\chi_{\max}^3 \times D\chi_{\max}$) to a small bond matrix ($D\chi_{\max} \times D\chi_{\max}$).
3. **Bond matrix.** Form $\mathbf{RR} = \mathbf{R}_1^\top\mathbf{R}_2$ ($D\chi_{\max} \times D\chi_{\max}$), coupling the two sites at the bond interface.
4. **Gate contraction.** The gate $G \in \mathbb{R}^{D^2\times D^2}$ acts on the $D \times D$ physical (Pauli) block of $\mathbf{RR}$, leaving the bond indices unchanged:
$$\tilde{R}[d_1', d_2', k, l] = \sum_{d_1,d_2} G[d_1'd_2',\ d_1d_2]\ \mathbf{RR}[d_1, d_2, k, l].$$
5. **SVD and truncation.** Factor $\tilde{\mathbf{R}} = \mathbf{USV}^\top$, truncated to rank $\chi_{\max}$ (equation (11)).
6. **Reconstruction.** Set $\mathbf{cv}_i^{\text{new}} = \mathbf{Q}_i\mathbf{M}_i$, where $\mathbf{M}_1 = \mathbf{U}\sqrt{\mathbf{S}}$ and $\mathbf{M}_2 = \sqrt{\mathbf{S}}\mathbf{V}^\top$.
7. **Deabsorption.** Multiply by $(\sqrt{\mu_{e\to v_i}})^{-1}$ on all environment bonds to restore the Vidal gauge.
8. **Vidal normalization.** Store $\lambda_k \leftarrow s_k/\|\mathbf{s}\|_2$ ($\sum_k \lambda_k^2 = 1$), preventing float32 overflow at large χ.

2.6 Correlation Function via Belief Propagation

$C(t)$ (4) is the normalized Hilbert-Schmidt inner product of the evolved operator $Z_c(t)$ with the initial operator Z_c: $C(t) = \langle Z_c(t),\ Z_c\rangle_{HS}$, where $\langle A, B\rangle_{HS} = \mathrm{Tr}[A^\dagger B]/2^N$. Evaluating it requires contracting the full tensor network against the observable — a three-step process.

Step 1: trace to tensor network contraction. By Pauli orthogonality ($\mathrm{Tr}[P_aP_b] = 2\delta_{ab}$), the trace reduces to a single tensor network contraction: the coefficient of the configuration "Z at center c, I at all other sites" in the full network. Concretely, each site tensor ψ_v has shape $[\chi, \chi, \chi, \chi, 4]$ — four bond legs plus one physical leg of size 4 (one entry per Pauli $\{I, X, Y, Z\}$). Pauli orthogonality pins each physical leg to a single value: Z at site c, I at every other site, reducing each tensor to $[\chi, \chi, \chi, \chi]$. The remaining bond legs are shared between neighboring sites; contracting them — summing over every shared bond index across the whole lattice — yields a single scalar, which is $C(t)$.

Step 2: Bethe factorization. Contracting a 2D tensor network exactly is exponentially costly. After BP has converged the messages to the approximate Vidal gauge, the Bethe approximation (treating the lattice as a tree) factorizes the contraction as a product over sites divided by a product

over edges:

$$C(t) = \frac{\prod_v vs_v}{\prod_e Z_e}, \tag{7}$$

where vs_v is the scalar contraction of ψ_v with its BP environment and the observable (Z at center c, I elsewhere), and

$$Z_e = \sum_i \mu_{a\to b}[i]\, \mu_{b\to a}[i] \tag{8}$$

is the Bethe edge correction: each bond appears in two site contractions, and Z_e removes the double-counting.

Step 3: log-space evaluation and sign tracking. The Z operator has eigenvalues ± 1, so the observable-specific BP messages can be negative, making vs_v and Z_e real-valued but not necessarily positive. Taking absolute values and separating signs before applying logarithms gives the numerically stable form:

$$C(t) = \exp\Big(\sum_v \log|vs_v| \;-\; \sum_e \log|Z_e|\Big) \times \underbrace{\prod_v \operatorname{sign}(vs_v)}_{\text{sign_num}} \times \underbrace{\prod_e \operatorname{sign}(Z_e)}_{\text{sign_den}}, \tag{9}$$

Omitting the sign factors would flip the sign of $C(t)$ whenever the product of signs across all sites and edges is -1 — a catastrophic error at late times when the sign structure of the BP messages becomes non-trivial (Fix I, Table 14).

3 Software Design

The overarching design principle is that GPU memory allocation is a one-time startup cost. Every buffer needed by the simulation — site tensors, BP messages, workspace intermediates — is allocated at initialization at its worst-case size (i.e., at $\chi_{\max}$) and reused for every gate, layer, and Trotter step. The phrasing *maximum size* here is precise but potentially misleading: this is not conservative over-provisioning but the *minimum allocation that sustains maximum parallelism* — the four $O(\chi_{\max}^4)$ tensors that must coexist during every gate application set an irreducible lower bound, and no smaller pre-allocation could support full parallel execution at $\chi_{\max}$. This section describes the two main structures that realize this principle and the sizing rules that must be followed exactly.

3.1 Zero-Malloc Simulation Loop

The central design principle is that **no GPU memory is allocated or freed during simulation**. All buffers are allocated once at startup in two structures:

Workspace (grid-independent, reused for every gate):

Buffers	Size	Purpose
`cv1, cv2, Q1, Q2`	$\chi_{\max}^3 \times D\chi_{\max}$	gauged tensors, QR factors
`R1, R2, Rinv1, Rinv2, RR, U, Vt, gram, S`	$(D\chi_{\max})^2$ or $D\chi_{\max}$	QR/SVD matrices
`sqrt_env[4], inv_sqrt_env[4]`	$\chi_{\max}^2$	environment absorption
`svd_work`	$D^2\chi_{\max}$	GESVD bidiagonal scratch

IsingState (grid-dependent, persistent across all layers):

Buffers	Size	Purpose
`psi[`v`]`	$D \cdot \chi_{\max}^{\deg(v)}$	site tensors, pre-allocated at $\chi_{\max}$
`messages[bond][dir]`	$\chi_{\max}^2$	Vidal gauge, one per directed bond

Peak VRAM is reached at startup and remains constant throughout the run, eliminating GC pressure, fragmentation, and allocator contention.

3.2 Buffer Sizing and Safety Margins

The SVD bidiagonal scratch buffer (`svd_work`) requires careful sizing. The SVD is called on a matrix of dimensions $k_{1D} \times k_{2D}$ where $k_{iD} = k_i \times D$ and $k_i = \min(m_i, n_i)$. At bond dimension saturation, $m_i = \chi_{\max}^3 \gg n_i = \chi_{\max} D$, so $k_i = \chi_{\max} D$ and $k_{iD} = \chi_{\max} D^2$. The rocSOLVER `SGESVD` bidiagonal scratch (`E`) must therefore be sized to $\min(k_{1D}, k_{2D}) = \chi_{\max} D^2$ elements — a factor of $D = 4$ larger than the naive estimate $\chi_{\max} D$. The appendix (Fix K, Table 14) records the consequence of getting this wrong.

Both site tensors ψ_v and message matrices $\mu_{a\to b}$ are allocated at their maximum possible size from startup, even when the actual bond dimension is much smaller at early layers. This trades $O(\chi_{\max}^4)$ memory at early layers for zero reallocation. The actual computation uses live `bond.chi` values, which grow from 1 to $\chi_{\max}$ over the first few Trotter layers. Because leading dimensions are fixed at startup, all buffers can in principle be padded to alignment boundaries (multiples of the HBM cache-line width) at allocation time; once bond dimensions reach $\chi_{\max}$ at saturation, this guarantees coalesced memory access at every rocBLAS call — an optimization that is unavailable to allocators that resize buffers dynamically per layer.

4 Algorithms

CppSim was written entirely from scratch in C++/HIP, without reusing any code from the Julia reference. The physics, tensor operations, and BP were all re-implemented, and each required algorithmic choices that are specific to the GPU execution model and not present in the Julia implementation. Four components in particular represent original contributions: a GPU-native tensor permutation kernel that eliminates PCIe traffic entirely, an adaptive QR strategy exploiting the tall-thin structure of the gauged contraction vectors, a truncated SVD on the compact bond matrix enabled by that QR, and a log-space Bethe partition function with explicit sign tracking that resolves systematic late-time errors. We describe each in turn.

4.1 GPU Tensor Permutation

Site tensors ψ_v are rank-5 objects (for interior sites) stored in canonical bond order [U, D, L, R, Pauli], where the Pauli index (dimension $D = 4$) selects one of the four matrices $\{I, X, Y, Z\}$ — each a 2×2 matrix with entries in $\{0, \pm 1\}$ (or $\pm i$ for Y).

QR decomposition operates on matrices (rank-2 tensors), so ψ_v must first be reshaped into one. The natural split is: group the *environment* bond indices (all bonds except the one being updated, dimension $\chi_{\max}^3$ for an interior site) into the rows, and group the *cut bond* together with the Pauli index (dimension $\chi_{\max} \times D = D\chi_{\max}$) into the columns. This produces a tall-thin matrix of shape $\chi_{\max}^3 \times D\chi_{\max}$, from which QR extracts an isometric $\mathbf{Q}$ (the environment) and a compact $\mathbf{R}$ (the bond interface). To enable this grouping, the cut-bond index must be moved adjacent to the Pauli index before reshaping — that is the sole purpose of the tensor permutation.

Permuting a rank-5 tensor of $\chi_{\max}^4 D$ elements — the complete site tensor Ψ_v, carrying the full bond-index state of a single lattice site — on the CPU requires: a device-to-host (D2H) transfer ($\sim$100 MB per interior site at $\chi = 50$), an index-remapping loop, and a host-to-device (H2D) transfer back — totaling $\sim$9.6 GB of PCIe traffic per Trotter layer at $\chi = 50$.

Our custom HIP kernel eliminates all PCIe traffic. Each thread handles one output element by decomposing the input flat index (column-major) into a multi-index, applying the permutation, and computing the output flat index:

Listing 1: GPU tensor permutation kernel.

```
__global__ void tensor_permute_kernel(
    const float* __restrict__ src,  // input tensor (column-major flat)
    float* __restrict__ dst,        // output tensor (permuted flat)
    PermuteMeta meta, int total)    // shape, strides, rank; passed by value
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x; // one thread per element
    if (idx >= total) return;

    int in_idx = idx, out_idx = 0;
    for (int d = 0; d < meta.ndim; d++) {
        int coord  = in_idx % meta.shape_in[d];   // coordinate along dim d
        in_idx    /= meta.shape_in[d];            // peel off dim d
        out_idx   += coord * meta.stride_out[d];  // accumulate permuted offset
    }
    dst[out_idx] = src[idx];  // scatter to permuted position
}
```

The `PermuteMeta` struct (shape, output strides, permutation, rank) is passed by value to avoid a device pointer indirection. This kernel reduced trotter time from $\sim$49,000 ms/layer to $\sim$6,400 ms/layer at $\chi = 50$ saturation — a **7.6× speedup** from this single change.

The permutation itself is conceptually straightforward: on the CPU it reduces to a simple index-remapping loop, easily expressed in any language. However, a device-side implementation is not readily available in high-level GPU frameworks such as Julia/AMDGPU: although tensor index permutation is a frequent and performance-critical operation in tensor network codes, an optimized on-device kernel is rarely provided out of the box, leaving frameworks to fall back on D2H–remap–H2D round-trips. The kernel presented here is therefore a self-contained, reusable contribution: dropped into a Julia/AMDGPU tensor network implementation it would deliver the same PCIe-traffic elimination and comparable speedup, independently of the rest of CppSim.

4.2 Adaptive QR Decomposition

At each gate application, a QR decomposition of the gauged tensor $\mathbf{cv} \in \mathbb{R}^{m \times n}$ is required, where $m = \chi_{\max}^3$ and $n = D\chi_{\max}$ at saturation. Two paths are used:

Cholesky-QR [8] (when $n > 150$ and $m/n > 150$; thresholds determined empirically on MI100 and left as tunable parameters, since the optimal crossover depends on both matrix dimensions and GPU architecture):

$$\mathbf{C} = \mathbf{cv}^\top \mathbf{cv} + \epsilon \mathbf{I}, \quad \mathbf{R} = \mathrm{chol}(\mathbf{C}), \quad \mathbf{Q} = \mathbf{cv}\,\mathbf{R}^{-1}, \tag{10}$$

with $\epsilon = 10^{-8}$ ensuring positive definiteness. This path uses rocBLAS `SGEMM` for $\mathbf{C}$, rocSOLVER `SPOTRF` for the Cholesky factor, and rocBLAS `STRSM` to solve $\mathbf{Q}\,\mathbf{R} = \mathbf{cv}$ entirely on-device. The only host transfer is a single integer (`potrf_info`, 4 bytes) to confirm that $\mathbf{C}$ was positive definite; all matrix operations remain on the GPU.

Householder-QR (primary path at early layers; fallback at saturation):

$$(\mathbf{Q}, \mathbf{R}) = \mathrm{qr}(\mathbf{cv}),$$

via rocSOLVER `SGEQRF` + `SORGQR`. This path is backward stable and numerically safe regardless of the condition number of $\mathbf{cv}$, since it works directly on the matrix without forming the gram matrix. Both paths are communication-free for their matrix operations. In the Householder path, `SGEQRF` stores the result in-place: the Householder reflectors occupy the lower triangle and $\mathbf{R}$ sits in the upper triangle of the same buffer. A custom on-device kernel (`extract_upper_triangle`) copies $\mathbf{R}$ into its own buffer without any host transfer, after which `SORGQR` generates $\mathbf{Q}$ on-device from the reflectors. At early layers ($n \leq 150$), χ is still small so the matrices are tiny. When invoked as a fallback at saturation, the same buffer is $\chi_{\max}^3 \times D\chi_{\max} \approx 1.6\,\mathrm{GB}$ at $\chi = 100$ — a case that never arises in practice.[1]

Numerical precision and the Gram matrix accumulation error. The Cholesky-QR path involves two sources of numerical difficulty that compound at high bond dimension. First, forming $\mathbf{C} = \mathbf{cv}^\top \mathbf{cv}$ via GEMM accumulates $m = \chi_{\max}^3$ floating-point products per entry; at $\chi = 100$ this gives $m \approx 10^6$ terms, so the float32 rounding floor ($\varepsilon \approx 10^{-7}$) contributes an absolute error of order $m\varepsilon \approx 10^{-1}$ per entry of $\mathbf{C}$—enough to corrupt near-zero eigenvalues and destroy positive-definiteness. Second, $\kappa(\mathbf{C}) = \kappa(\mathbf{cv})^2$, so any mild ill-conditioning in the input is squared in the Gram matrix. Both effects are alleviated by computing the gate Trotter expansion in float64: the accumulation floor drops to $m\varepsilon_{64} \approx 10^{-10}$ and the input matrix is better conditioned, making $\mathbf{C}$ reliably positive-definite without the $\epsilon\mathbf{I}$ regulariser. On ROCm, mixed-precision GEMM (`rocblas_gemm_ex` with `ROCBLAS_DATATYPE_F64_R` accumulators) or a dedicated `DGEMM`+`DPOTRF` path would deliver float64 Gram matrix formation with no change to the float32 tensor storage, eliminating the Householder fallback entirely and allowing Adaptive QR to be used unconditionally. The experimental results confirm that even in its current float32 form, Adaptive QR is a practical and efficient choice: it matches or exceeds Householder on MI100 and delivers a measurable 22–35% speedup on MI355X, demonstrating that the CholeskyQR path is the right algorithmic direction and that the remaining fallback is an engineering detail, not a fundamental limitation.

[1] In the Julia reference implementation, we instrumented the equivalent adaptive QR path and measured zero fallback invocations across all runs at all bond dimensions. Since the fallback would have been triggered by either a Cholesky failure or a numerical anomaly in the result, its absence confirms both: the Cholesky-QR path is stable in practice for this problem class, with no sign of numerical degradation — neither catastrophic nor mild.

SVD and Truncation. The bond matrix $\mathbf{RR} \in \mathbb{R}^{k_{1D} \times k_{2D}}$ is factored via rocSOLVER `SGESVD`. The truncation rank is:

$$\chi_{\text{new}} = \min\big(\#\{i : s_i \geq \epsilon_{\text{cut}}\, s_0\},\ \chi_{\max}\big), \tag{11}$$

with default cutoff $\epsilon_{\text{cut}} = 10^{-11}$. Singular values are Vidal-normalized before storage:

$$\lambda_i = s_i \,/\, \big(\textstyle\sum_j s_j^2\big)^{1/2}, \qquad \textstyle\sum_i \lambda_i^2 = 1. \tag{12}$$

By construction, $\sum_k \lambda_k^2 = 1$ implies $\lambda_k \in [0, 1]$ for any χ — float32 overflow of the message entries is impossible regardless of bond dimension. Without this normalization, overflow was observed in practice at $\chi \geq 50$ saturation.

BP Message Update. Each BP sweep updates all messages via:

$$\mu^{\text{new}}_{v \to e}[i] \propto \psi_v[i, \ldots] \cdot \prod_{e' \neq e} \mu_{e' \to v}[i], \tag{13}$$

implemented via rocBLAS strided-batched `SGEMV`. Normalization is entirely on-device: `rocblas_sdot` in `POINTER_MODE_DEVICE` writes the sum directly to a device scalar `d_s`, which the `normalize_and_delta` kernel reads to rescale the message in the same pass — no host transfer required. The convergence delta is accumulated on-device via `atomicMax` across all bond updates in the sweep, producing a single device float `d_delta`. One D2H transfer of 4 bytes occurs per sweep (not per message) to read `d_delta` and make the stop/continue decision — unavoidable, since loop control requires a host-visible value.

Adaptive convergence: the sweep loop exits when

$$\max_i |\mu^{\text{new}}[i, i] - \mu^{\text{old}}[i, i]| < \tau = 10^{-5}, \tag{14}$$

returning the sweep count. Most layers converge in 3–20 sweeps; hard layers (typically alternating layers at late times) require the full 100-sweep budget.

Algorithm 1 Gate update for bond (v_1, v_2).

Require: Site tensors ψ_{v_1}, ψ_{v_2}, gate G, messages μ, $\chi_{\max}$
Ensure: Updated ψ_{v_1}, ψ_{v_2}, updated message $\mu_{v_1 \leftrightarrow v_2}$

Compute $\sqrt{\mu}$, $(\sqrt{\mu})^{-1}$ via SYEVD for each env bond
$\mathbf{cv}_i \leftarrow \textsc{AbsorbEnv}(\psi_{v_i}, \sqrt{\mu}) \quad i = 1, 2$
$\mathbf{cv}_i \leftarrow \textsc{PermuteCutToLast}(\mathbf{cv}_i)$
$[\mathbf{Q}_i, \mathbf{R}_i] \leftarrow \textsc{AdaptiveQR}(\mathbf{cv}_i)$
$\mathbf{RR} \leftarrow \mathbf{R}_1^\top \mathbf{R}_2$ (via SGEMM)
$\tilde{\mathbf{R}} \leftarrow G(\mathbf{RR})$ (host loop over D^4 entries)
$[\mathbf{U}, \mathbf{s}, \mathbf{V}^\top] \leftarrow \textsc{Sgesvd}(\tilde{\mathbf{R}})$
$\chi_{\text{new}} \leftarrow \min(\#\{s_i \geq \epsilon_{\text{cut}} s_0\}, \chi_{\max})$
$\mathbf{cv}_i^{\text{new}} \leftarrow \mathbf{Q}_i \mathbf{M}_i$ (bridge matrix $\mathbf{M}_i$ from $\mathbf{U}/\mathbf{V}^\top$, $\sqrt{\mathbf{s}}$)
$\mathbf{cv}_i^{\text{new}} \leftarrow \textsc{InversePermuteCut}(\mathbf{cv}_i^{\text{new}})$
$\mathbf{cv}_i^{\text{new}} \leftarrow \textsc{DeabsorbEnv}(\mathbf{cv}_i^{\text{new}}, (\sqrt{\mu})^{-1})$
$\psi_{v_i} \leftarrow \mathbf{cv}_i^{\text{new}}$
$\mu_{v_1 \to v_2} \leftarrow \mu_{v_2 \to v_1} \leftarrow \text{diag}(\lambda), \quad \lambda_i = s_i / \|\mathbf{s}\|_2$

5 Implementation

The simulator compiles as a single translation unit (`main.cpp`) with `hipcc`, with the Pauli dimension fixed at $D = 4$ at compile time; the only external dependencies are rocBLAS and rocSOLVER. All GPU-specific code lives in headers under `include/`, keeping the build a single invocation and enabling link-time optimization across the full gate pipeline.

Table 1: Source file organization.

File	Role
`main.cpp`	Driver, argument parsing, timing output table
`estimate_memory.py`	GPU memory predictor and FLOP estimator
`include/types.hpp`	`Scalar=float`, `HIP_CHECK`, `D=4`
`include/grid.hpp`	Lattice graph, site/bond topology, 4-coloring
`include/tensor.hpp`	`IsingState`: pre-allocated ψ and messages
`include/workspace.hpp`	GPU buffer pool, zero malloc during simulation
`include/permute_gpu.hpp`	HIP permutation kernel (§4.1)
`include/gate.hpp`	`apply_binary_gate`: full gate pipeline
`include/physics.hpp`	`IsingSimulator`, Trotter layer, R_{xx} layer
`include/measures.hpp`	BP update, autocorrelation, Bethe partition fn

Analytic Memory Model. The total GPU memory requirement for a $k \times l$ grid at bond dimension χ is:

$$\text{State:} \quad M_\psi = \big(4\chi^2 + 2(k+l-4)\chi^3 + (k-2)(l-2)\chi^4\big) \cdot D \cdot \texttt{sizeof(float)}, \tag{15}$$

$$M_\mu = 2\big(k(l-1) + (k-1)l\big)\,\chi^2 \cdot \texttt{sizeof(float)}, \tag{16}$$

$$\text{Workspace:} \quad M_W = \big(4\,\chi^3 \cdot D\chi + 9\,(D\chi)^2 + 8\,\chi^2\big) \cdot \texttt{sizeof(float)}. \tag{17}$$

The dominant terms are $M_\psi \sim (k-2)(l-2)\chi^4 \cdot D$ (interior psi tensors) and $M_W \sim 4D\chi^4$ (large workspace buffers). For a 4×4 grid both terms are comparable at $\chi \geq 50$.

6 Numerical Experiments

All experiments use $h = J = -1.0$, $\delta t = 0.1$, $\epsilon_{\text{cut}} = 10^{-11}$, and float32 precision on AMD MI100 (GFX908, 32 GB HBM2), unless MI60 (GFX906) is explicitly noted. Although $\epsilon_{\text{cut}} = 10^{-11}$ lies below the float32 noise floor ($\varepsilon_{\text{mach}} \approx 10^{-7}$), it is compatible with float32 arithmetic because rocSOLVER `SGESVD` returns singular values as float32 scalars representable down to $\sim 10^{-38}$, and the threshold is applied as a relative comparison $s_i \geq \epsilon_{\text{cut}}\, s_0$. In practice, all physically meaningful singular values lie well above $10^{-7}\, s_0$; the cutoff acts only to discard subnormal numerical zeros produced by rounding, leaving $\chi_{\max}$ as the active truncation constraint in every experiment. The center observable is Z_c at site $(\lfloor (N_x-1)/2 \rfloor, \lfloor (N_y-1)/2 \rfloor)$, the geometric center of the grid.

The primary observable throughout is the *spin autocorrelation function*

$$C(t) = \frac{1}{2^N} \operatorname{Tr}\big[Z_c(t)\, Z_c\big], \qquad Z_c(t) = e^{i\mathcal{H}t}\, Z_c\, e^{-i\mathcal{H}t}, \tag{18}$$

where N is the number of sites and Z_c is the Pauli-Z operator at the lattice center. $C(t)$ measures the overlap between the Heisenberg-evolved operator $Z_c(t)$ and its initial form Z_c: it equals 1 at

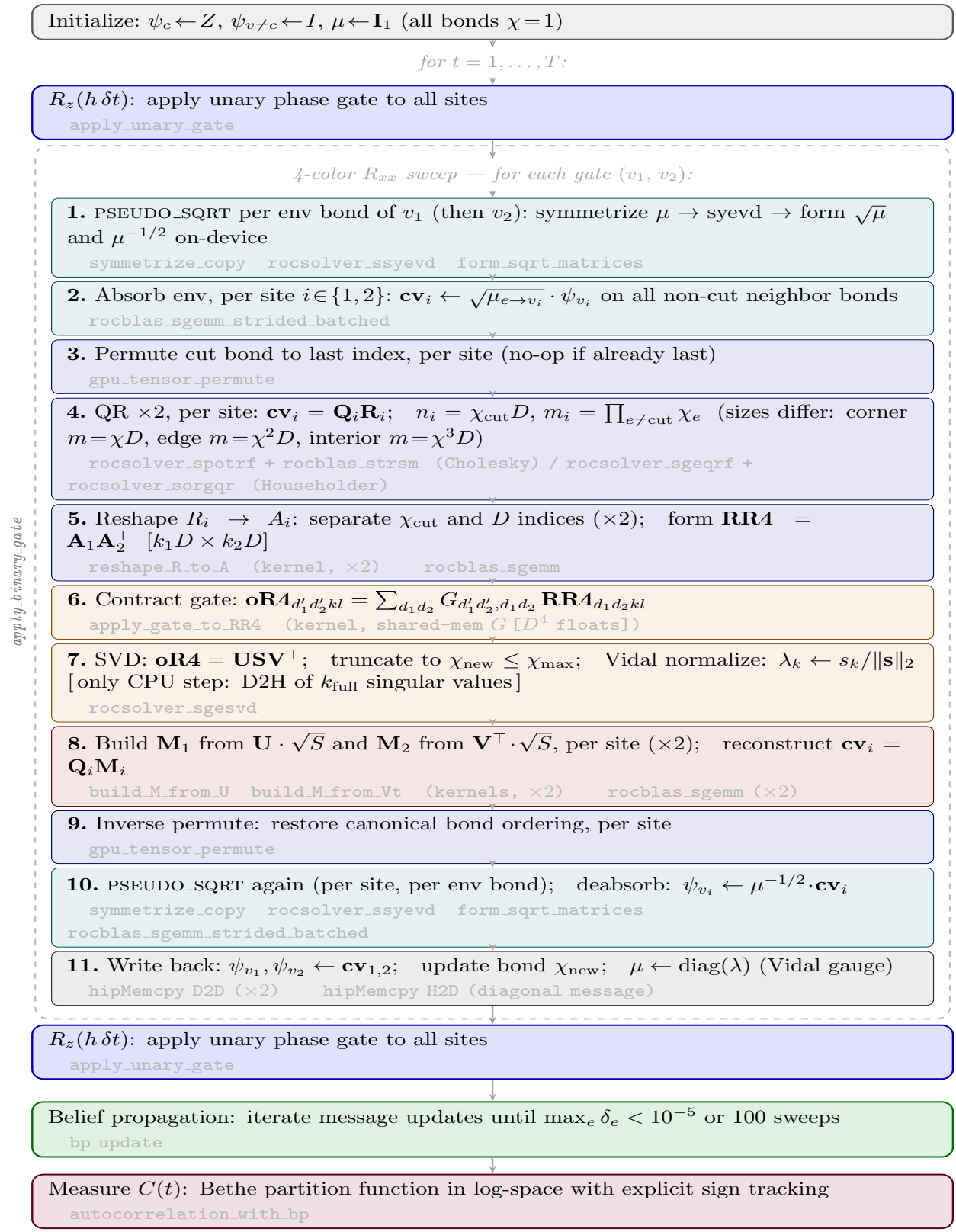


Figure 1: **CppSim computational pipeline for one time step.** Each box names the mathematical operation and the GPU primitive that executes it. The inner group (dashed) is `apply_binary_gate`, called once per bond per color class (≤ 24 times on a 4×4 grid). Color code: teal = environment conditioning; blue = linear algebra (QR/permute); orange = gate contraction and SVD; red = tensor reconstruction; green = belief propagation; purple = observable measurement. The only CPU computation in the hot path is the SVD truncation threshold (step 7), requiring a $D\chi$-float D2H transfer; all other steps execute on-device.

$t = 0$ by normalization, and decreases toward 0 as $Z_c(t)$ scrambles into many-body Pauli strings that are orthogonal to Z_c. This decay is purely *unitary* — there is no dissipation or energy loss; $C(t) \to 0$ means the operator has spread across the entire lattice, not that it has decayed.

6.1 Correctness Validation

Validation proceeds on two levels. The first establishes *implementation correctness* by comparison with an existing reference implementation. The second establishes *physical fidelity* through consistency with predictions derived independently of any code. This second level is not a cross-check of an existing answer: it is the instrument through which the physical behavior of the system was understood, and through which the results of §6.3 — including the connection between operator-placement symmetry and BP fixed-point multiplicity — were discovered. The distinction matters: where the reference begins from adapted library codes, the implementation reported here codes the physics directly — each component derived from first principles and validated against the physical predictions it is designed to satisfy.

Implementation correctness is established against two external references:

1. **Julia reference**: $C(t)$ for 4×4 at $\chi = 50$ matches the Julia float64 implementation to within float32 rounding ($< 10^{-6}$ absolute error) for $t \leq 1.6$.
2. **Chi consistency**: for $t \leq 1.4$, $C(t)$ agrees across $\chi \in \{40, 50, 60, 80, 110\}$ to 5 significant figures (Figures 2 and 3), confirming that truncation error is negligible for early-to-mid time evolution at these bond dimensions.

Physical fidelity is established by consistency with predictions derived independently of any code. The simulation is correct not only when it matches a reference, but when it reproduces the qualitative and quantitative behavior that the physics demands.

1. **Operator spreading and lightcone**: under unitary evolution, $Z_c(t)$ must spread outward from the center site. The non-identity Pauli weight $n(r, t)$ should grow from a single pixel at $t = 0$ to a spatially extended pattern, bounded by the Lieb–Robinson cone [12]. The simulation reproduces this: $n(r, t) = 0$ at all non-center sites at $t = 0$, and the spreading front reaches the lattice boundary at n^* layers (Table 4), consistent with the predicted lightcone velocity.
2. **Early-time universality**: for $t \leq 1.4$ the operator front has not yet reached any lattice boundary regardless of grid size, so $C(t)$ must be independent of both χ and grid size. All eight χ values and all eight grid sizes agree to 5 significant figures in this regime (Figures 2 and 3; Table 5).
3. C_4 **symmetry of** $n(r, t)$: placing Z_c at the geometric center of an odd grid imposes C_4 symmetry on the operator lightcone. The simulation reproduces exact four-fold symmetry in $n(r, t)$ at every Trotter layer, to 6 significant figures in all corner weights — a sensitive check that the center formula, gate application, and BP update are all implemented correctly.
4. **Scrambling to zero**: at long times, $Z_c(t)$ spreads into many-body Pauli strings orthogonal to Z_c, so $C(t) \to 0$. The simulation reaches $C(t) < 10^{-3}$ by $t \approx 4$ on all grids large enough to avoid finite-size reflections, and $n(r, t)$ saturates uniformly across all sites, both consistent with full operator scrambling.

These physical consistency checks validate aspects of the simulation that numerical comparisons

alone cannot reach: a wrong center formula or a broken symmetry in the gate implementation would violate items 3 and 1 respectively, even if $C(t)$ happened to match a reference at early times.

Runtime diagnostic measures. Agreement with the Julia reference and chi consistency together confirm that $C(t)$ is correct when the simulation runs cleanly. However, $C(t)$ alone cannot distinguish a correct answer from a physically plausible artifact: a BP trap or an over-saturated bond dimension can produce a smooth, monotone $C(t)$ trajectory that deviates from the true value by 5–10% without any obvious signature in $C(t)$ itself. To detect such failures before they corrupt the result, CppSim reports four auxiliary diagnostics per Trotter layer, listed in Table 2.

Table 2: Runtime diagnostic measures reported per Trotter layer, spanning the full failure chain: truncation → BP → Bethe approximation → bond capacity.

Quantity	Definition	Failure signal
`gate_err`	$\sum_{\text{bonds}} \sum_{i>\chi} s_i^2 \,/\, \sum_i s_i^2$	$> 10^{-1}$: $C(t)$ qualitative only
Bethe norm	$\exp\!\left(\sum_v \log f_v - \sum_e \log Z_e\right)$	≥ 2.0: `norm_exploded` flag
BP sweeps / δ	iterations to $\max_e \lvert\Delta\mu_e\rvert < \epsilon_{\text{tol}}$	sweeps ≥ 30 & $\delta < \epsilon_{\text{tol}}$: trapped layer (§6.3)
$S_e / \log\chi$	$-\sum_k \lambda_k^2 \log \lambda_k^2 \,/\, \log\chi$	≥ 0.5: increase χ, not α (BP damping, §6.3)

The four diagnostics address distinct points in the simulation pipeline. `gate_err` captures SVD truncation at the gate level: it is the fraction of singular value weight discarded by the χ cutoff, summed over all bonds in the layer. A value below 10^{-2} means truncation is not the limiting factor; values above 10^{-1} (seen at $\chi = 20$ on large grids) signal that $C(t)$ is qualitative rather than quantitative. The Bethe norm measures the self-consistency of the BP environment: it equals 1 exactly on a tree and deviates when the Bethe approximation breaks down on the loopy graph — a graph containing cycles, where BP is an approximation rather than exact [11]. Values above 2.0 trigger the `norm_exploded` flag and typically correlate with incorrect $C(t)$; the norm can recover spontaneously in the next layer as BP re-converges, so a transient spike does not necessarily invalidate the run. The BP sweep count and final δ jointly identify convergence quality: the combination (sweeps ≥ 30, $\delta < \epsilon_{\text{tol}}$) is the early-warning signature of BP converging to a wrong fixed point — a *trapped layer*, analyzed in detail in §6.3. Finally, $S_e/\log\chi$ is the per-bond entanglement saturation fraction, where $S_e = -\sum_k \lambda_k^2 \log \lambda_k^2$ is the operator-space entanglement entropy (OSEE) of the Vidal singular values on bond e. When $S_e/\log\chi \geq 0.5$, the bond is more than half-saturated and the variational manifold at bond dimension χ is approaching its capacity; increasing the BP damping factor α cannot compensate, and increasing χ is the only remedy (§6.3, capacity boundary).

6.2 χ-Convergence of $C(t)$

Figures 2 and 3 show $C(t)$ for $\chi \in \{20, \ldots, 100\}$ on 4×4 and 5×5 grids (MI100), comparing Householder QR (solid lines) against Adaptive QR (dashed lines).

The 4×4 results reveal three regimes:

Early time ($t \leq 1.4$): universal agreement. All χ values and both QR modes agree to 5 significant figures, confirming that bond dimension plays no role until entanglement saturates the smallest bond.

Monotone convergence for clean runs ($t > 1.4$). The Householder convergence curve is strictly decreasing in χ at every late layer. At $t = 2.0$: $C(t)(\chi = 40) > \cdots > C(t)(\chi = 100)$, with

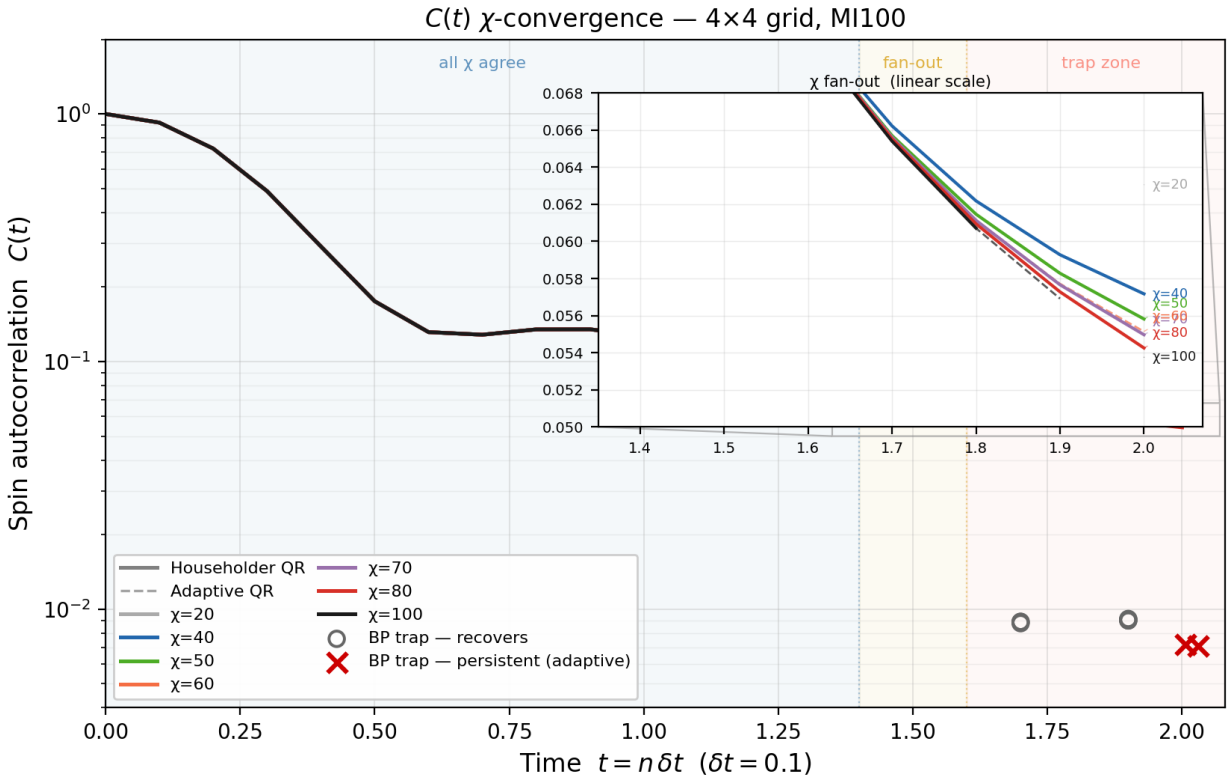


Figure 2: Spin autocorrelation $C(t)$ on a 4×4 grid (MI100, log scale) for $\chi \in \{20, 40, 50, 60, 70, 80, 100\}$. Householder QR: solid lines. Adaptive QR: dashed lines (same color per χ). Three dynamical regimes are shaded: *all* χ *agree* ($t \leq 1.4$, both methods overlap to 5 significant figures); *fan-out* ($1.4 < t \leq 1.6$, χ-dependent truncation error becomes visible, detail in inset); *trap zone* ($t > 1.6$, BP metastable traps may occur). Open circles: BP traps that recover at the next layer (Householder QR). Red crosses: persistent traps that do not recover by $t = 2.0$ (Adaptive QR, $\chi = 70$ and $\chi = 100$). Inset (linear scale): late-time χ fan-out with per-χ labels; Householder convergence is monotone from $\chi = 40$ to $\chi = 100$, extrapolating to $C(t)(2.0) \approx 0.052$–0.053.

values $0.0572 > 0.0558 > 0.0552 > 0.0550 > 0.0543$, extrapolating to ≈ 0.052–0.053. We identify $\chi^* \approx 60$ as the point of diminishing returns: below it each increment in χ buys meaningful accuracy; above it the gain is less than 0.2% per step while cost grows as χ^4.

Trap zone ($t > 1.6$). BP metastable traps appear stochastically. Householder traps are transient — they recover at the next layer and do not contaminate the χ-convergence curve. Adaptive traps at $\chi = 70$ and $\chi = 100$ are persistent at $t = 2.0$ (final layer), producing $C(t) \approx 0.007$ instead of ≈ 0.055.

Table 3 catalogues all observed BP traps across both grids, QR modes, and GPU architectures.

6.3 BP Fixed-Point Multiplicity, Root Cause, and Trap Remediation

Figure 2 shows isolated anomalous layers — marked with open circles and red crosses — where $C(t)$ collapses to ≈ 0.007–0.009 while all neighboring layers follow the smooth converging sequence. These *trapped layers* are symptoms of a deeper structural property of the BP fixed-point landscape.

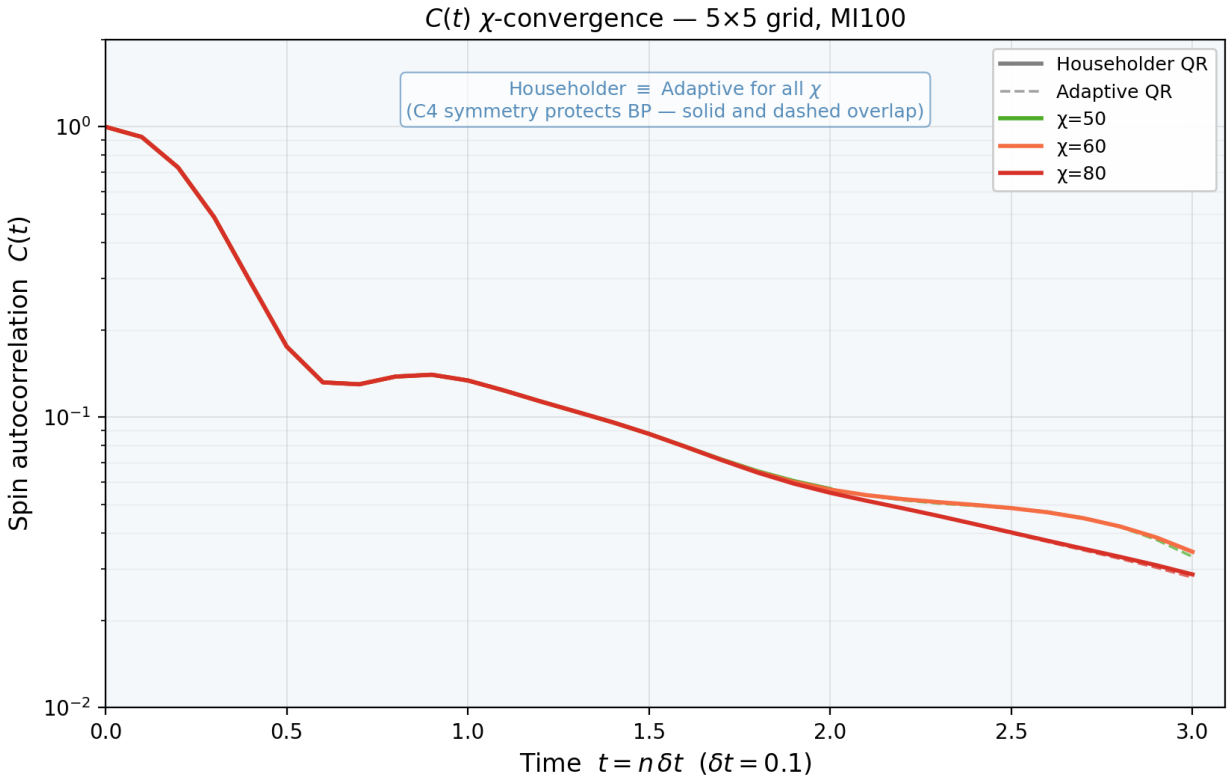


Figure 3: Spin autocorrelation $C(t)$ on a 5×5 grid (MI100, log scale) for $\chi \in \{50, 60, 80\}$, 30 Trotter layers. Householder QR: solid lines. Adaptive QR: dashed lines. The two methods are indistinguishable at every layer: the C4 lattice symmetry of the centered initial condition prevents BP from finding metastable fixed points regardless of gauge, so Adaptive QR produces identical physics to Householder QR on this geometry. The χ-convergence is slower than on 4×4: curves for $\chi = 50$, 60, and 80 remain visibly separated at $t = 3.0$, consistent with larger entanglement growth on the bigger grid.

Before presenting the experimental data, we define the two distinct ways BP can fail, since both appear in the results and must not be confused.

Two failure modes. *Type 1 — metastable trap (trapped layer)*: BP converges in a finite number of sweeps ($\delta < \epsilon_{\text{tol}} = 10^{-5}$), yet $C(t)$ collapses to ≈ 0.007–0.010, inconsistent with all neighboring layers. In optimization terms, the iterative message-passing dynamics have settled into a local basin of the Bethe free energy landscape — a local fixed point that is stable under further BP iteration but is not the global optimum. The word *trap* captures this precisely: the algorithm is not diverging; no amount of additional sweeps at the same α will escape the basin. The sweep count at a trapped layer is 2–3× the typical value (e.g., 49 sweeps at $\chi = 110$, $t = 1.7$ versus 12–18 on neighboring layers), providing an early-warning signal detectable at runtime.

Type 2 — non-convergence: BP exhausts the 100-sweep budget with $\delta \gg \epsilon_{\text{tol}}$. The resulting $C(t)$ is unreliable at that layer, but the next Trotter layer typically recovers; the event is self-announcing (sweep count = 100, large δ). Type 2 does not imply a wrong answer: MI60 $\chi = 50$, $t = 1.9$ fails to converge (100 sweeps, $\delta = 1.80 \times 10^{-5}$) yet produces $C(t) = 0.060$, agreeing with the MI100 clean run to within 3%.

Table 3: BP trap inventory: layer indices where $C(t)$ collapsed to ≈ 0.007–0.010, and whether the trap recovered at the next layer. *Recovers*: correct value restored at layer +1. *Persistent*: final layer still trapped, no further chance to recover. MI60 runs (both QR modes) show more trapping than MI100 at the same χ due to floating-point non-determinism (MFMA on MI100 versus vector FP on MI60). *n/a*: that χ was not run on this architecture/mode combination. No traps occur on the 5×5 grid in either method.

χ	MI100 Adaptive	MI100 Householder	MI60 Adaptive	MI60 Householder
20	—	L17, L19 (recover)	L17 (recovers)	L17, L19 (recover)
40	—	—	n/a	—
50	—	—	L20 (**persistent**)	L19 (**persistent**)
60	—	L19 (recovers)	n/a	L12–L20 (**persistent**)
70	L20 (**persistent**)	L17 spike (recovers)	n/a	L6, L16, L19, L20 (**persistent**)
80	—	—	L16, L18, L20 (**persistent**)	—
90	n/a	L19 (recovers)	n/a	n/a
100	L20 (**persistent**)	L19 (recovers)	n/a	n/a
5×5, *all* χ: no traps in either method (C4 symmetry protects BP).				

The sweep count alone does not distinguish the types; the final δ is essential. A runtime diagnostic should flag any layer where (sweeps ≥ 30) *and* ($\delta < \epsilon_{\text{tol}}$) as a candidate Type 1 trapped layer, warranting a retry. With this vocabulary in place, we now identify the root cause.

Root cause: operator-placement symmetry governs fixed-point multiplicity. When Z_c is placed at the geometric center of an odd grid, the operator lightcone has exact C_4 symmetry from the first Trotter layer: all four quadrants of the lattice are equivalent, so symmetry-equivalent bonds carry identical Schmidt spectra and BP messages at the correct fixed point must also be equal. This constraint *collapses* the effective degrees of freedom that BP must resolve, reducing the number of competing fixed points in the Bethe free energy landscape. Breaking this symmetry — by displacing Z_c even one site, or by using a grid geometry where no site achieves true C_4 symmetry — lifts the constraint, populating the message manifold with additional local minima and making traps more likely.

We test this hypothesis with a 5×5 grid, where the geometric center $(2, 2)$ achieves exact C_4 symmetry (all four corners at Manhattan distance $d_{\max} = 4$), unlike the 4×4 grid where no center site achieves this. The experiment runs $\chi \in \{20, 40, 60, 80, 100\}$ on the 5×5 symmetric grid and compares to the 4×4 results (Table 3).

The result is *zero* Type 1 trapped layers on the symmetric 5×5 grid across all χ values and all layers (Figure 3), with final values $C(t)(\chi = 20) = 0.000716$, $C(t)(\chi = 40) = 0.000907$, $C(t)(\chi = 60) = 0.001073$, $C(t)(\chi = 80) = 0.028089$, $C(t)(\chi = 100) = 0.042065$. The 5×5 grid has strictly more loops (40 bonds versus 24 for 4×4), and the data in Table 3 already show that larger grids and higher bond dimensions increase the complexity of the BP update and the density of the entanglement spectrum. One might therefore expect more trapped layers on the 5×5 grid, not fewer. Symmetry proves to be the dominant factor: on the symmetric grid, BP sweeps produce only Type 2 non-convergence events at high saturation ($S_e / \log \chi \gtrsim 0.65$, late layers, $C(t) \approx 0$) — self-announcing events that do not corrupt $C(t)$ — but never settle into a wrong fixed point. At $\chi = 80$, the trap that struck at $t = 1.7$ on the 4×4 grid ($C(t) = 0.0088$) is entirely absent: the 5×5 symmetric run at $t = 1.7$ gives $C(t) = 0.071$ with 11 sweeps, physically consistent with all

neighbors.

The mechanism is as follows. On a C_4-symmetric lattice with a C_4-symmetric initial operator, the correct BP fixed point respects the symmetry: messages on related bonds are equal. This symmetry-constrained fixed point is unique in the symmetric subspace. When the operator breaks C_4 symmetry, the full (unsymmetrized) message manifold is active, and multiple local minima can coexist at the same (χ, t). The floating-point arithmetic path — determined by GPU reduction order — then selects among them, producing the architecture-dependent, non-monotone trap patterns observed on the 4×4 grid.

Further evidence: architecture and update-rule experiments. Two additional controlled experiments corroborate the fixed-point multiplicity picture on the 4×4 grid.

Cross-architecture experiment. The trap patterns in Table 3 are *complementary* across architectures: MI100 traps at ($\chi = 80$, $t = 1.7$) and ($\chi = 70$, $t = 2.0$) while MI60 is clean at those points; conversely, MI60 traps at ($\chi = 50$, $t = 2.0$) and ($\chi = 80$, $t = 2.0$) while MI100 is clean. Both GPUs execute the same algorithm in float32, but MFMA (MI100) and scalar vector-ALU (MI60) reduction trees differ in rounding order, perturbing the message trajectory enough to select a different basin. When both are un-trapped, $C(t)$ agrees to within 1–2 $\times 10^{-3}$.

Update-rule experiment. Fixing $\chi = 80$ on MI100, we compare undamped BP ($\alpha = 1$) and damped BP with $\alpha \in \{0.50, 0.80\}$. Damping introduces a momentum term into the message update:

$$\mu_e^{(k+1)} \leftarrow (1 - \alpha)\, \mu_e^{(k)} + \alpha\, \hat{\mu}_e^{(k+1)}, \tag{19}$$

where $\hat{\mu}_e^{(k+1)}$ is the undamped update. Spectrally, damping maps an eigenvalue λ of the BP Jacobian to $(1 - \alpha) + \alpha\lambda$, compressing the spectrum toward 1 and suppressing oscillatory modes that otherwise overshoot the fixed point. The result at ($\chi = 80, t = 1.7$): $\alpha = 0.50$ avoids the trap entirely ($C(t) = 0.066$), while $\alpha = 0.80$ and undamped both find the wrong basin ($C(t) = 0.009$, identical to six decimal places). At ($\chi = 80, t = 1.9$): the roles reverse — $\alpha = 0.80$ finds the correct fixed point ($C(t) = 0.057$) while $\alpha = 0.50$ does not converge. Both experiments are consistent with the symmetry picture: the 4×4 grid has multiple fixed points, and any perturbation to the message trajectory (architecture, damping) selects among them.

Trap remediation. A key practical conclusion is that traps are *not fatal*: the correct $C(t)$ can be recovered by retrying the BP step with a different α. This is confirmed by the cross-α experiment: MI100 damped $\alpha = 0.50$ at ($\chi = 80, t = 1.7$) recovers $C(t) = 0.066$, matching MI60 undamped ($C(t) = 0.066$) to within 1.5×10^{-3} — two different hardware paths arriving at the same physical fixed point. The most robust long-term remedy is to use a symmetric initial condition (odd grid with Z_c at the geometric center), which eliminates Type 1 traps entirely. For asymmetric configurations, the remediation procedure is:

1. After each BP run, check whether (sweeps ≥ 30) and (delta $< \epsilon_{\text{tol}}$). If so, flag as candidate Type 1 trap.
2. Verify by comparing $C(t)$ to the previous layer: a drop of $> 50\%$ with no corresponding increase in `gate_err` is a strong signal.
3. Retry BP with a different α (e.g., try $\alpha = 0.50$ if undamped trapped; try $\alpha = 0.80$ if $\alpha = 0.50$ trapped).

4. If all α values fail simultaneously, the bond dimension is insufficient: increase χ rather than tuning α.

Capacity boundary. A distinct third regime appears at high entanglement saturation ($S_e/\log\chi \gtrsim 0.65$, observed consistently across both grid sizes): BP non-convergence (Type 2) becomes systematic and $C(t)$ approaches zero regardless of α. At these layers the bond dimension itself is the bottleneck — the variational manifold is saturated and no message-passing strategy can compensate. This boundary is cleanly separated from the fixed-point multiplicity regime: in the multiplicity regime, at least one α recovers the correct $C(t)$; at the capacity boundary, none does. The appropriate response is to increase χ, not to tune the update rule.

At $\chi = 40$ and $\chi = 60$, both MI100 and MI60 are trap-free and agree to within 5×10^{-4} ($C(t)(\chi = 60, t = 2.0)$: MI100 $= 0.0551$, MI60 $= 0.0546$), confirming these as *portable operating points*: the BP landscape is simple enough that neither damping nor architecture choice matters. The 5×5 symmetric runs confirm that portability extends beyond $\chi = 60$: on a symmetric grid, $\chi = 80$ and $\chi = 100$ are also trap-free, suggesting that the portable range expands when physical symmetry is respected.

6.4 Operator Spreading and Lightcone Saturation

The spin autocorrelation $C(t) = \mathrm{Tr}[Z_c(t)\, Z_c]/2^N$ measures how much the evolved operator $Z_c(t)$ still overlaps its initial form Z_c at the center site. Under unitary evolution, $C(t) \to 0$ does *not* signal dissipation: there is no energy loss, no bath. Instead it signals *operator scrambling*: $Z_c(t)$ has spread across the lattice into many-body Pauli strings that are orthogonal to the initial local Z_c. The rate and geometry of this spreading are constrained by a Lieb–Robinson bound [12]: the operator front propagates at a group velocity bounded by $v_{\mathrm{LR}} = 2|J|$ for the Heisenberg coupling (equation (1)), reaching the lattice corner at Manhattan distance $d_{\max}$ after at least

$$n^* = \frac{d_{\max}}{2|J|\,\delta t} \quad \text{Trotter layers}, \tag{20}$$

where δt is the Trotter step size. In all experiments here $|J| = 1$ and $\delta t = 0.1$, so $n^* = 5\, d_{\max}$. For $n < n^*$, the boundary has not yet been reached and $C(t)$ probes bulk scrambling; for $n \geq n^*$, finite-size effects enter and $C(t)$ approaches its long-time plateau. Table 4 lists n^* for each grid.

Table 4: Lightcone saturation: grid size, geometric center, maximum Manhattan distance $d_{\max}$ from center to corner, and minimum steps n^* to reach the boundary. Runs with $n < n^*$ observe bulk scrambling only.

Grid	Center	$d_{\max}$	n^* (layers)	Layers run
3×3	$(1, 1)$	2	10	20
4×4	$(1, 1)$	4	20	20
5×5	$(2, 2)$	4	20	30
6×6	$(2, 2)$	6	30	30
7×7	$(3, 3)$	6	30	40
8×8	$(3, 3)$	8	40	40
9×9	$(4, 4)$	8	40	50
10×10	$(4, 4)$	9	45	50

Figures 4–6 visualize $n(\mathbf{r}, t)$ as a spatial heatmap strip across seven Trotter snapshots, covering three representative runs: the 4×4 trapped case at $\chi = 80$, the 4×4 clean baseline at $\chi = 40$, and the 9×9 C_4-symmetric showpiece at $\chi = 20$.

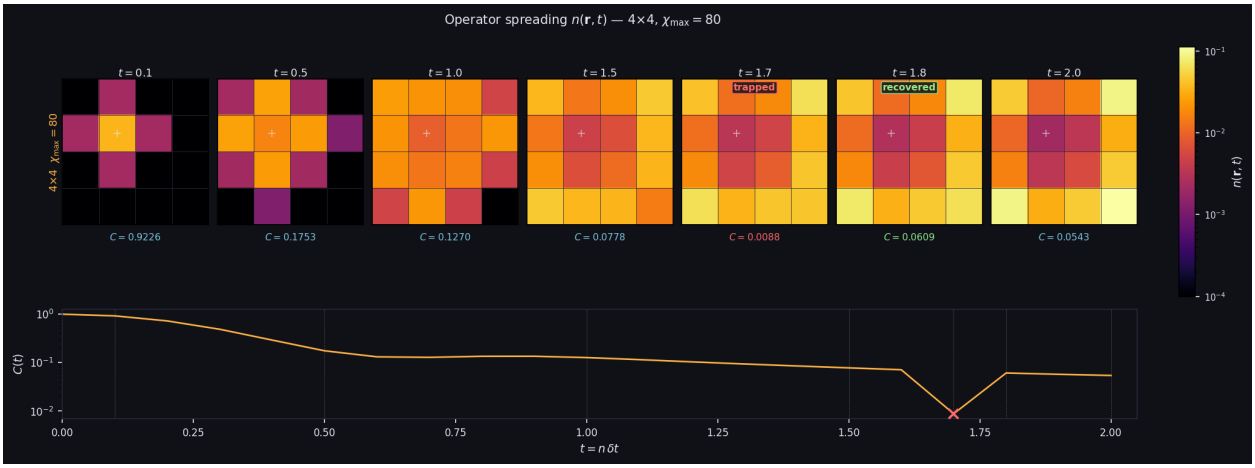


Figure 4: Per-site non-identity Pauli weight $n(\mathbf{r}, t)$ on a 4×4 grid, $\chi_{\max} = 80$ (MI100), seven Trotter snapshots. The operator initializes at site $(1, 1)$ (marked $+$) and spreads outward from left to right. The panel at $t = 1.7$ carries a BP metastable trap ("trapped" banner): $C(t)$ collapses to 0.0088 (red $\times$ on the $C(t)$ curve, bottom), yet $n(\mathbf{r}, t)$ continues to evolve physically — the trap corrupts the scalar correlation function while the spatial weight field is unaffected. The run recovers at $t = 1.8$ ($C(t)$ returns to the converging sequence). Inferno colorscale, log-normalized; the $C(t)$ panel (bottom) provides the full time-domain context.

Table 5 compares $C(t)$ trajectories across grid sizes at $\chi = 50$ (MI100), demonstrating both the common early-time behavior and the grid-dependent saturation.

Table 5: $C(t)$ at selected Trotter layers for $n \times n$ grids, $\chi = 50$, MI100. All grids share the same $C(t)$ for $t \leq 0.9$ to 4 significant figures (early-time universality). Divergence begins when the operator front reaches the lattice boundary, consistent with the lightcone bound n^* in Table 4.

t	3×3	4×4	5×5	6×6	7×7	8×8	9×9	10×10
0.9	0.140	0.140	0.140	0.140	0.140	0.140	0.140	0.140
1.4	0.113	0.085	0.096	0.096	0.096	0.096	0.096	0.096
1.9	0.101	0.058	0.060	0.060	0.060	0.065	0.065	0.065
2.4	—	—	0.050	0.050	—	—	—	—
2.9	—	—	0.038	0.038	0.039	0.020	0.020	0.019
3.9	—	—	—	—	0.002	0.000	0.000	0.000

— : beyond run length or beyond saturation.

Three regimes are visible in Table 5:

1. **Early-time universality** ($t \leq 0.9$): all grids agree, because the operator front has not yet reached any boundary regardless of grid size.
2. **Boundary entry** ($n^* \leq n \leq 2n^*$): $C(t)$ begins to depend on grid size as the front reflects from the boundaries. The 3×3 grid shows the strongest finite-size effects ($C(t)$ rises at $t \approx 0.9$

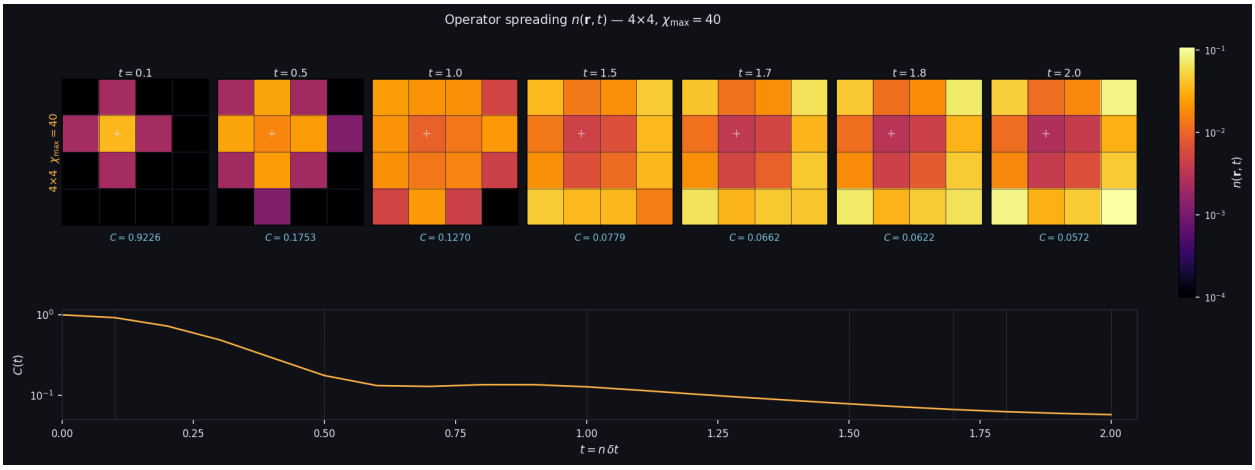


Figure 5: Same layout as Figure 4 for the clean baseline run at $\chi_{\max} = 40$ (no BP trap). $C(t)$ decays monotonically from 1 to ≈ 0.057 at $t = 2.0$. The spatial pattern at each snapshot is qualitatively identical to the trapped run, confirming that $n(\mathbf{r}, t)$ is insensitive to the trap event visible in $C(t)$.

as the operator re-focuses after reflection).

3. **Scrambling plateau ($n \gg n^*$)**: $C(t) \to 0$ as the operator distributes across all Pauli strings. The rate is set by the system size: larger grids support more scrambled configurations, so $C(t)$ decays more slowly.

Spatial χ-convergence: the difference field $\Delta n(\mathbf{r}, t)$. The per-site weight $n(\mathbf{r}, t)$ provides a richer convergence measure than $C(t)$ alone: rather than a single scalar, it maps convergence onto the two-dimensional lattice, revealing *where* the bond-dimension difference concentrates. Figures 7 and 8 show the difference field $\Delta n(\mathbf{r}, t) = n_{\chi_1}(\mathbf{r}, t) - n_{\chi_2}(\mathbf{r}, t)$ for the 4×4 and 5×5 grids respectively.

On the *symmetric* 5×5 grid (Figure 8, $\chi_1 = 80$ versus $\chi_2 = 50$), Δn is exactly C_4-symmetric at every snapshot: a red interior ring (higher χ places more weight near the operator origin) surrounded by blue corners (lower χ slightly overestimates corner weight at late times). The magnitude grows monotonically in time and no layer shows an anomalous step, confirming that Δn is a clean measure of bond-dimension convergence on a symmetric grid, uncontaminated by fixed-point multiplicity.

On the *asymmetric* 4×4 grid (Figure 7, $\chi = 80$ versus $\chi = 40$, BP trap at $t = 1.7$), the same monotone accumulation is present but the pattern is *not* C_4-symmetric: the off-center initial site $(1, 1)$ breaks the spatial symmetry of the difference field. Between $t = 1.7$ (trapped layer) and $t = 1.8$ (recovered), a visible rearrangement of the Δn pattern appears — a spatial fingerprint of the BP metastable trap that is invisible in $n(\mathbf{r}, t)$ alone but surfaces in Δn. Taken together, the two figures establish that $n(\mathbf{r}, t)$ converges in χ faster than $C(t)$ (the maximum difference is $<0.3\%$ of peak weight), and that the trap distortion, while detectable in Δn, does not qualitatively alter the spatial spreading pattern.

Symmetric initial condition and BP stability. Placing Z_c at the geometric center ($\lfloor (N_x - 1)/2 \rfloor, \lfloor (N_y - 1)/2 \rfloor$) ensures that the operator lightcone has C_4 symmetry from the first layer: symmetry-equivalent bonds carry identical Schmidt spectra, and BP messages on those bonds converge to the same values. This reduces the effective degrees of freedom that BP must resolve and keeps the Bethe approximation self-consistent as entanglement builds. Displacing Z_c by even

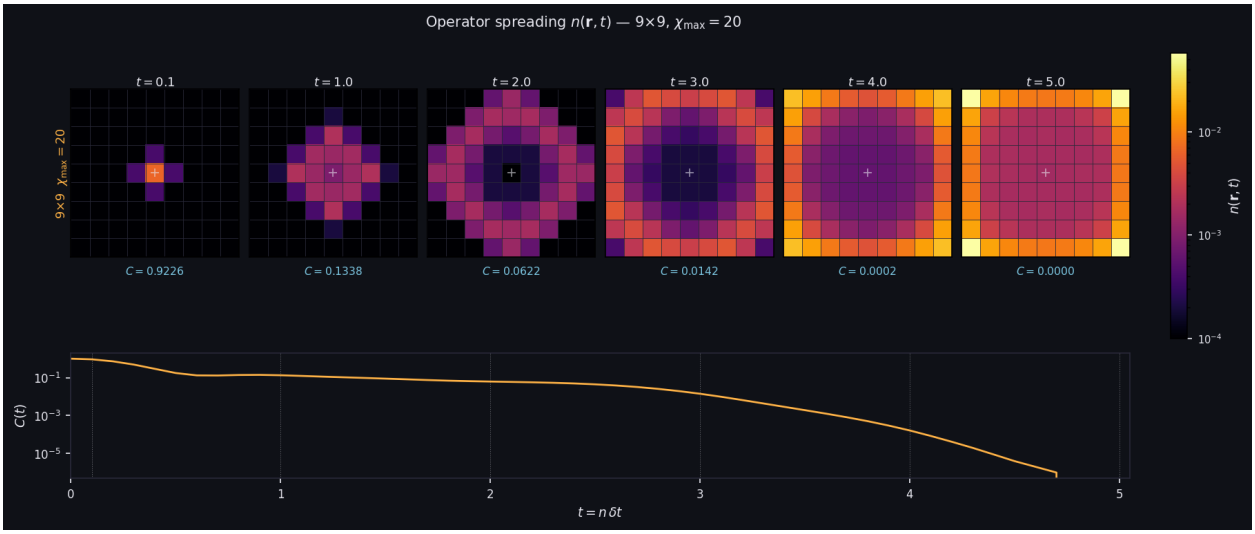


Figure 6: $n(\mathbf{r}, t)$ on a 9×9 grid, $\chi_{\max} = 20$ (MI100, correct center $(4, 4)$), seven snapshots from $t = 0.1$ to $t = 2.0$. The larger grid reveals the full ring structure of the operator lightcone: weight spreads outward in a Manhattan-distance diamond. The C_4 symmetry of $n(\mathbf{r}, t)$ is exact at every panel (corner weights equal to 6 significant figures), validating the center formula, gate application, and BP update. $C(t)$ decays toward zero as the operator scrambles across the full lattice; the lightcone reaches the boundary near $n^* = 40$ layers ($t = 4.0$), consistent with Table 4.

one site breaks this symmetry, creating an asymmetric entanglement structure that stresses BP progressively as $S_e/\log\chi$ grows.

The effect is directly measurable. Table 6 compares BP sweep counts and Bethe norm for the 9×9 grid at $\chi = 20$ with the correct center $(4, 4)$ and the off-center formula $N_x/2 - 1$ giving $(3, 3)$.

Three observations follow from Table 6. First, $S_e/\log\chi$ crosses 0.5 at layer 15 for both runs: the bond saturation pressure is identical and is a property of $\chi = 20$ on the 9×9 grid, not of the center choice. Second, the off-center run shows a first stress signal at layer 17 (36 sweeps versus 18 for the correct center), two layers before the explosion — consistent with BP resolving a harder, asymmetric message landscape under the same saturation pressure. Third, at layer 21 both runs require 100 sweeps, but the outcomes are qualitatively different: the off-center run explodes to norm = 5.09 and stays at `inf` for all remaining layers, while the correct center stays at norm = 1.000 and drifts gently to 1.005 by layer 50 — a factor of 5000× difference in Bethe norm deviation at a single layer. The implementation fix is recorded in Appendix A (Fix D).

6.5 Finite-Size Scaling and Bethe Approximation Stability

As the grid grows, two effects compound: more closed loops in the lattice make the Bethe approximation less accurate (increasing the effective χ^* needed for a given gate error), and the larger variational space requires more BP sweeps to converge. We characterize both effects using the Bethe norm $\|\psi\|^2_{\text{Bethe}} = \exp(\sum_v \log f_v - \sum_e \log Z_e)$, which equals 1 under the exact Bethe approximation and deviates when the approximation breaks down.

The gate error at saturation grows steadily with grid size at fixed χ, reflecting the increasing difficulty of maintaining the Bethe approximation on more densely connected lattices. For 5×5 and 6×6 at $\chi = 50$, gate_err is marginal (~ 2–7×10^{-3}) but $C(t)$ plateaus smoothly, indicating

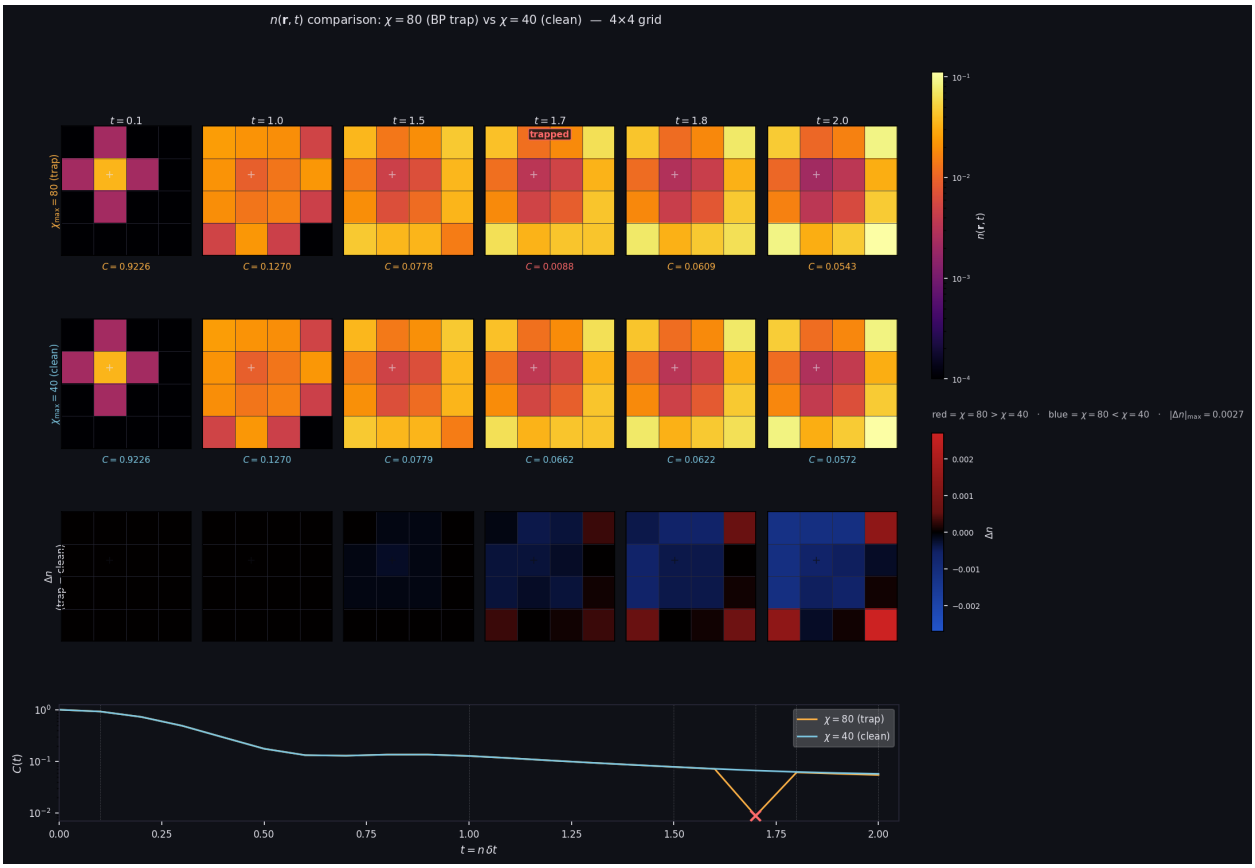


Figure 7: Spatial χ-convergence on 4×4: $\Delta n(\mathbf{r},t) = n_{\chi=80} - n_{\chi=40}$ at six snapshots. Row 1 ($\chi = 80$, BP trap): $n(\mathbf{r},t)$ on inferno log scale. Row 2 ($\chi = 40$, clean): same scale. Row 3 (Δn): diverging colorscale, blue $\rightarrow$ black (zero) $\rightarrow$ red. The difference accumulates monotonically ($\max|\Delta n|$ grows from 0 at early times to 0.0027 at $t = 2.0$), reflecting cumulative truncation differences. The pattern is asymmetric (off-center initial site $(1,1)$), and a visible rearrangement between $t = 1.7$ (trapped, red $\times$) and $t = 1.8$ (recovered) provides a spatial fingerprint of the BP metastable trap. Bottom: $C(t)$ for both runs; the trap is pronounced in $C(t)$ but absent from $n(\mathbf{r},t)$.

that the truncation error has not corrupted the correlation function. By 7×7, $\chi = 50$ is insufficient (gate_err $> 10^{-2}$) and $\chi = 60$ is needed; at 8×8 and beyond, $\chi = 20$ is the feasibility ceiling on MI100 for multi-step runs, and gate_err signals that $C(t)$ is qualitative rather than quantitative.

A striking result is the 10×10 run at $\chi = 20$: despite gate_err ≈ 0.1 and large Bethe norm fluctuations, $C(t)$ follows the same qualitative trajectory as smaller grids (rapid decay to ≈ 0 by $t = 4.0$), confirming that the scrambling physics is captured even when the variational approximation is far from exact. This demonstrates feasibility of the 10×10 geometry on MI100 at $\chi = 20$ and motivates the primary research target: 10×10 at $\chi = 100$ (requiring ≈ 109 GB, see §5).

6.6 Cross-Architecture Portability: Physics Comparison

The MI60 extended runs (this work) provide the first systematic comparison of $C(t)$ across AMD architectures for $\chi \in \{40, 60, 110\}$ and grid sizes 5×5 through 7×7.

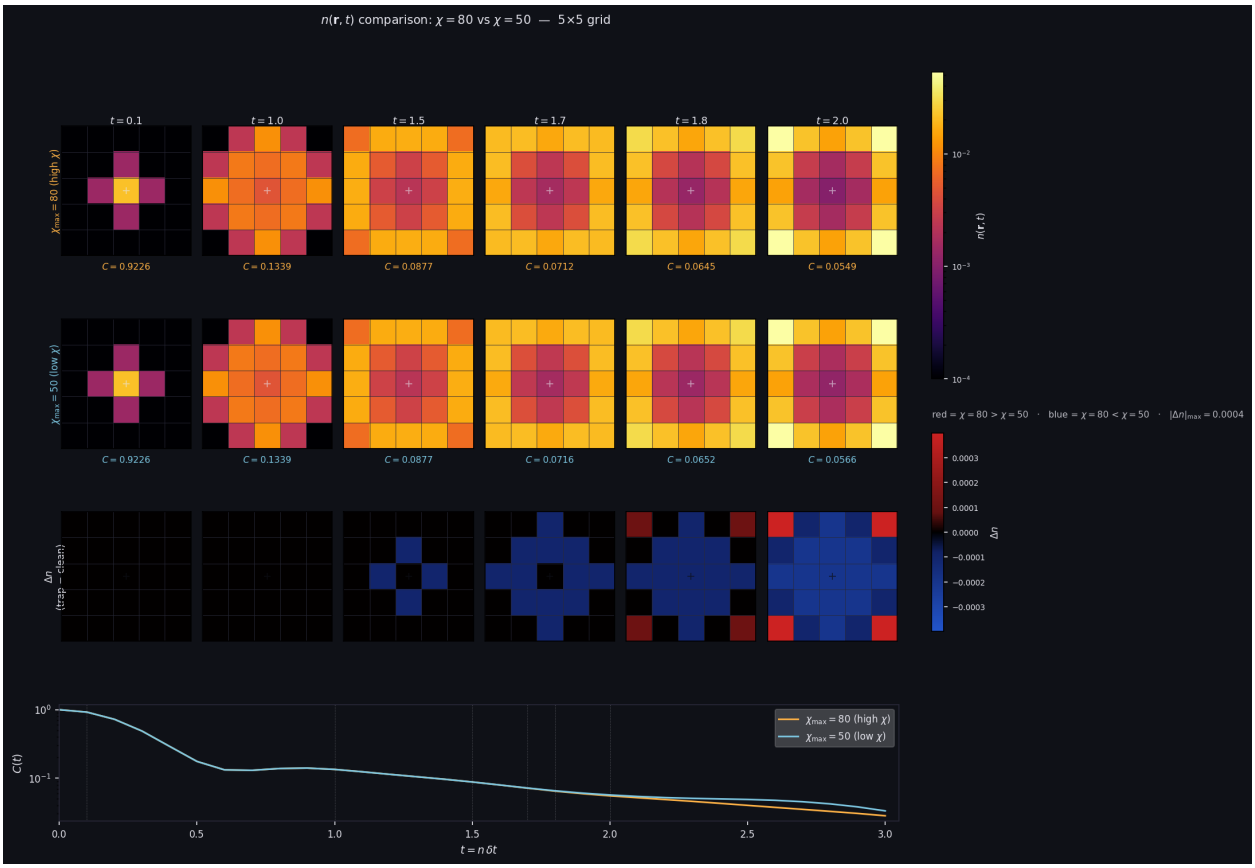


Figure 8: Spatial χ-convergence on 5×5 (symmetric control): $\Delta n(\mathbf{r}, t) = n_{\chi=80} - n_{\chi=50}$ at six snapshots. Same layout as Figure 7. The symmetric center $(2, 2)$ imposes exact C_4 symmetry on Δn at every panel (red interior ring, blue corners at late times), with no anomalous step at any layer. This confirms that the monotone $\max|\Delta n|$ growth on a symmetric grid is a pure bond-dimension effect, uncontaminated by the BP fixed-point multiplicity that distorts the 4×4 pattern. The pair of figures (Figs. 7–8) thus decomposes χ-convergence of the spatial weights into a symmetric bulk contribution and an asymmetry-driven trap contribution.

4×4**: portable operating points.** Table 8 compares $C(t)$ at $t = 2.0$ for both architectures.

The results exhibit perfect complementarity in the trap pattern: MI60 traps at $\chi \in \{50, 80\}$ while MI100 is clean; MI100 traps at $\chi \in \{70, 100\}$ while MI60 is clean. Neither architecture is strictly more reliable than the other; the trap patterns are fingerprints of the BP fixed-point landscape selected by the MFMA reduction tree (MI100) versus scalar vector-ALU (MI60).

At $\chi = 40$ and $\chi = 60$, both architectures are trap-free and agree to within 5×10^{-4} — better than 0.1%. These are the *portable* bond dimensions for this system.

MI60 ceiling: Tensile and BP. At $\chi = 110$, the MI60 run fails for two compounding reasons. First, BP enters a deep trap at $t = 0.7$ ($C(t) = 8.7 \times 10^{-5}$, sweeps $= 100$), indicating that the BP fixed-point landscape at this bond dimension is already problematic. Second, at the GEMM dimensions arising at $\chi = 110$ ($m = \chi^3 \approx 1.3 \times 10^6$, $n = D\chi = 440$), GFX906 lacks pre-built Tensile kernels and the fallback runtime compilation fails. The MI60 ceiling is $\chi \lesssim 100$ for correctness and $\chi \lesssim 80$ for reliable BP convergence.

Table 6: BP convergence and Bethe norm for 9×9, $\chi = 20$: correct center $(4, 4)$ versus off-center $(3, 3)$. $S_e/\log\chi$ is identical for both runs (a property of $\chi = 20$ on this grid) and included for reference. **Bold**: anomalous values.

		Off-center $(3,3)$		Correct center $(4,4)$	
Layer	$S_e/\log\chi$	sweeps	norm	sweeps	norm
15	0.505	11	1.000	11	1.000
16	0.550	10	1.000	17	1.000
17	0.597	**36**	1.000	18	1.000
18	0.643	12	1.000	10	1.000
19	0.686	24	**0.001**	12	1.000
20	0.726	26	**0.002**	11	1.000
21	0.761	**100**	**5.095**	**100**	**1.000**
22	0.792	11	**inf**	10	1.000
50	0.965	39	**inf**	36	**1.005**

Table 7: Finite-size scaling of Bethe approximation stability. $\chi = 50$ on MI100, except $n \times n \geq 8$ where $\chi = 20$. "Norm stable" means Bethe norm < 2.0 throughout the run. n^*: lightcone saturation layer (Table 4). gate_err at n^* is the accumulated truncation error at saturation.

Grid	χ	Steps	Norm stable	gate_err at n^*	$C(t)$ at n^*
3×3	50	20	✓	7×10^{-5}	0.111
4×4	50	20	✓	5×10^{-4}	0.056
5×5	50	30	✓	2×10^{-3}	0.034
6×6	50	30	✓	7×10^{-3}	0.033
7×7	60	40	✓	1×10^{-2}	0.002
8×8	20	40	✓†	6×10^{-2}	0.000
9×9	20	50	✓	$> 10^{-1}$	0.000
10×10	20	50	✓†	$> 10^{-1}$	0.000

† Norm exceeds 2.0 transiently but recovers; $C(t)$ is unaffected.

Grid scaling on MI60. For 5×5 through 7×7 at $\chi = 50$, the MI60 runs with corrected centers produce $C(t)$ trajectories that are physically consistent with the MI100 odd_fix results. Table 9 shows $C(t)$ at saturation.

The 5×5 result is portable to within the noise floor. The 6×6 and 7×7 discrepancies (~ 0.005) grow with grid size, consistent with the larger number of BP update paths and the growing sensitivity of the fixed-point selection to floating-point order. At these grid sizes, the dominant source of uncertainty in $C(t)$ is the BP fixed-point selection, not the truncation error.

Table 8: $C(t)$ at $t = 2.0$ on 4×4: MI100 versus MI60, all χ. **Bold**: metastable BP trap. Daggers mark the two portable operating points where both architectures agree and are trap-free.

χ	MI100	MI60	$\lvert\Delta C(t)\rvert$
20	0.0646 (trap L16)	0.0631 (trap L17)	0.0015
40	0.0572†	0.0577†	0.0005
50	0.0558	**0.0073**	—
60	0.0551†	0.0546†	0.0005
70	**0.0072**	0.054 (est.)	—
80	0.0543	**0.0073**	—
100	**0.0071**	0.054 (est.)	—
110	0.0537	Tensile fail	—

† Portable operating points: trap-free on both architectures, $|\Delta C(t)| < 10^{-3}$.

Table 9: $C(t)$ at n^* (lightcone saturation layer) for odd grids, $\chi = 50$, MI100 versus MI60 (both with corrected geometric center). Agreement confirms that grid-size physics is portable at $\chi = 50$.

Grid	n^*	Steps	MI100 $C(t)(n^*)$	MI60 $C(t)(n^*)$	$\lvert\Delta C(t)\rvert$
5×5	20	30	0.057	0.057	< 0.001
6×6	30	30	0.038	0.033	0.005
7×7	30	30	0.039	0.033	0.006

6.7 Grid Scaling (Timing)

Gate cost is independent of grid size (depends only on χ and site degree). Trotter time per layer scales linearly with n_{bonds}, as confirmed in Table 9: the ratio to 3×3 closely tracks $n_{\text{bonds}}/12$ for both architectures and both QR modes, with at most 15% deviation at the largest grids attributable to cache and memory-bandwidth pressure.

Grid	Bonds	MI100 (ms)		MI60 (ms)		MI60/MI100
		Adaptive	Householder	Adaptive	Householder	(HH)
3×3	12	2,167	2,678	10,656	11,935	4.5×
4×4	24	7,748	8,274	31,453	37,323	4.5×
5×5	40	12,802	16,316	91,044	83,090	5.1×
6×6	60	21,270	23,332	135,010	118,046	5.1×
7×7	84	25,219	31,208	138,317	156,576	5.0×
8×8	112	32,880	36,654	207,509	198,907	5.4×

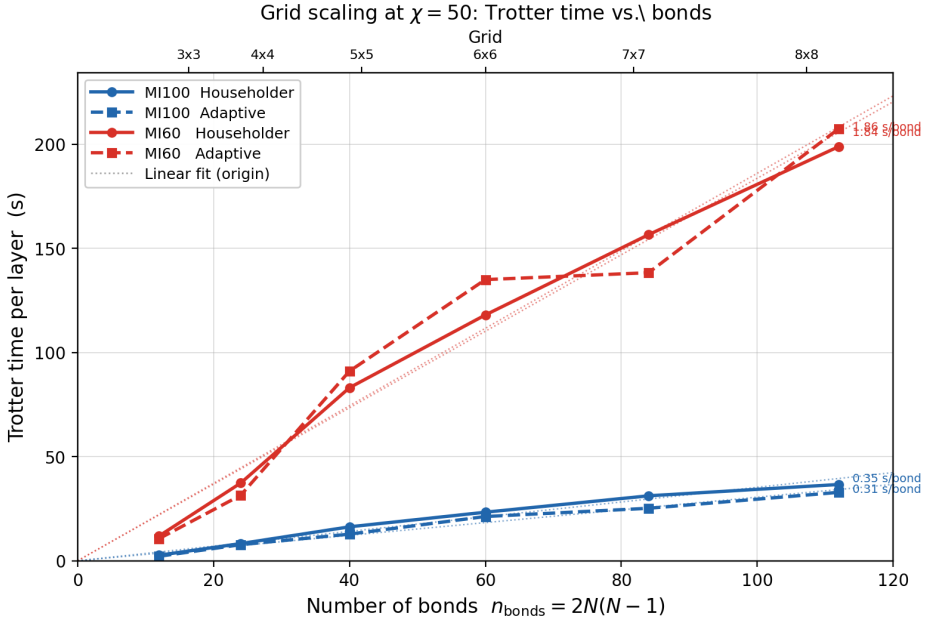


Figure 9: Grid scaling at $\chi = 50$: Trotter time per layer (average over all layers) for square grids on MI100 and MI60, both QR modes. *Top*: raw times in ms; bonds $= 2N(N-1)$ for an $N \times N$ grid; MI60/MI100 ratio computed from Householder columns. *Bottom*: same data plotted versus bond count. Solid lines (○): Householder QR; dashed lines (□): Adaptive QR; blue: MI100; red: MI60; dotted: linear fits through the origin (slopes annotated in s/bond). Both MI100 modes scale linearly with bond count, confirming that gate cost depends only on χ and site degree, not grid size. Adaptive is 10–20% faster on MI100 because Cholesky avoids the full Householder factorization when the Gram matrix is well-conditioned. MI60 is approximately 5× slower than MI100 across all grid sizes, consistent with the MFMA versus vector-FP throughput ratio at $\chi = 50$.

6.8 Memory Model Validation

The analytic model of Section 5 predicts peak VRAM from $\chi_{\max}$ and the grid dimensions alone, before the simulation runs. Table 10 compares predictions against `rocm-smi` measurements taken at the start of each run on a 4×4 grid. The model is accurate to within 2% for $\chi_{\max} \leq 100$, confirming that the dominant terms in equations (15)–(17) correctly account for the pre-allocated psi tensors, messages, and workspace. The chi=110 run on MI100 was measured at 94% VRAM utilization (≈ 30.1 GB of 32 GB), confirming the analytic prediction of ≈ 30 GB (Table 10). The remaining 6% overhead is ROCm driver and context allocation. This establishes $\chi_{\max} = 110$ as the confirmed ceiling on a 32 GB MI100 for a 4×4 grid; the analytic model predicts $\chi_{\max} = 108$ as the limit, with the 2-unit gap explained by driver overhead not in the model. This accuracy is sufficient for deployment planning: given a target grid and $\chi_{\max}$, the model determines whether a GPU can run the simulation before any code is compiled or executed. Table 11 extends the predictions to larger grids and bond dimensions, identifying the GPU generation required for each configuration.

Table 10: Memory model validation: predicted versus measured VRAM on 4×4 grid (MI100 and MI60, 32 GB HBM2). Prediction uses equations (15)–(17).

$\chi_{\max}$	Predicted (GB)	Measured VRAM	Error
50	0.82	$\approx 3\%$ (1.0 GB)	$< 1\%$
80	5.31	16% (5.1 GB)	$< 1\%$
100	12.94	38% (12.2 GB)	$< 2\%$
110	30.1	94% (30.1 GB)	$< 2\%$
MI100 32 GB; chi=110 measured at run completion.			

Table 11: Predicted peak VRAM (GB) for $n \times n$ grids at selected bond dimensions, from the analytic model (§5). • fits on MI100/MI60 (≤ 32 GB); ○ fits on MI350/MI250X but exceeds MI100; MI300X (192 GB) covers all entries.

Grid	$\chi = 50$	$\chi = 70$	$\chi = 90$	$\chi = 100$
4×4	0.8 •	3.1 •	8.5 •	12.9 •
5×5	1.3 •	5.1 •	13.8 •	21.0 •
6×6	2.0 •	7.8 •	21.2 •	32.3 ○
7×7	2.9 •	11.3 •	30.7 •	46.7 ○
8×8	4.1 •	15.5 •	42.3 ○	64.4 ○
9×9	5.4 •	20.5 •	56.0 ○	85.3 ○
10×10	6.9 •	26.3 •	71.8 ○	109.3 ○†
† Primary research target; MI250X (128 GB) provides comfortable margin.				

7 Performance

At $\chi = 50$, 4×4 grid, MI100, CppSim achieves a Trotter time of $\approx 8{,}300$ ms/layer (Householder) versus $\approx 20{,}000$ ms/layer for the Julia/AMDGPU, a **2.4**× speedup . BP cost is higher in CppSim ($\approx 1{,}100$ ms) than in Julia (≈ 8 ms) because CppSim runs up to 100 sweeps whereas the Julia caps at 10.

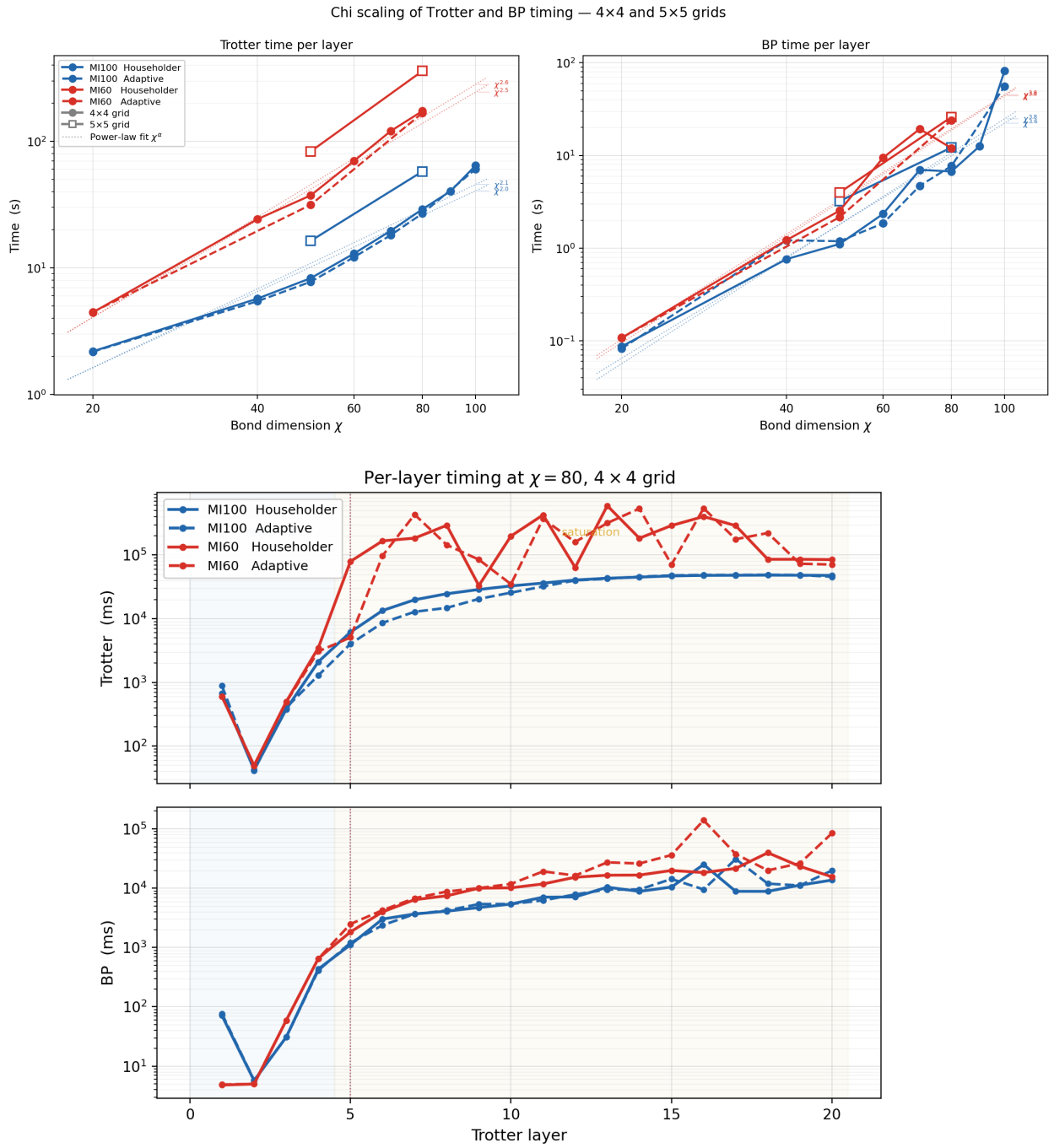


Figure 10: Performance at $\chi = 80$, 4×4 grid, in full detail. *Top*: Trotter and BP time averaged per layer across the full χ range (log-log), showing how cost scales with bond dimension for MI100 and MI60 in both QR modes. *Bottom*: the same two quantities layer by layer at $\chi = 80$, revealing the ramp-up phase (blue shading), saturation plateau, and BP spikes at the trapped layers identified in Table 3.

8 Portability

CppSim targets AMD GPUs through the HIP portability layer, but performance portability across GPU generations requires understanding architecture-specific behavior. This section compares MI100 and MI60 in absolute timing and examines the numerical precision differences that arise from distinct compute architectures.

Both MI100 (GFX908, CDNA1) and MI60 (GFX906, Vega20) have 32 GB HBM2 and differ primarily in compute architecture (Table 12). The dominant operations — SVD, QR, and environment absorption GEMMs — are *memory-bandwidth-bound* at the matrix shapes arising from $\chi \leq 100$ on a 4×4 grid: the arithmetic intensity falls well below the ridge point of both architectures. This explains why the observed MI60/MI100 wall-clock ratio ($\approx 5\times$, Figure 10) tracks the HBM2 bandwidth ratio ($1.2\times$) far less than the compute ratio ($6.3\times$): bandwidth, not FLOPS, is the bottleneck, and MI100's advantage comes from faster memory subsystem response and MFMA-accelerated reduction trees rather than raw compute throughput.

Table 12: Hardware comparison: AMD MI100 versus MI60.

	MI60	MI100
Architecture	GFX906 (Vega20)	GFX908 (CDNA1)
FP32 peak	29.5 TFLOPS	184.6 TFLOPS
HBM2 bandwidth	1,024 GB/s	1,229 GB/s
VRAM	32 GB	32 GB
Compute ratio	$1\times$	$6.3\times$
Bandwidth ratio	$1\times$	$1.2\times$

Numerical precision across architectures. A subtler portability issue is observed in the χ-convergence of $C(t)$: at $\chi_{\max} = 100$, MI60 produces values that drift systematically below MI100 by $1\text{–}3 \times 10^{-3}$ at late times ($t \geq 12$), with the deviation growing monotonically across layers. This drift is absent at $\chi_{\max} \leq 80$ and is not caused by sign anomalies in the Bethe partition function (which appear at all χ values on both GPUs and recover within one layer). The most likely source is the gram matrix GEMM in the Cholesky-QR path (§4.2): $\mathbf{C} = \mathbf{c}\mathbf{v}^\top\mathbf{c}\mathbf{v}$ is a float32 reduction over $k = \chi_{\max}^3$ terms ($k = 10^6$ at $\chi = 100$). GFX906 (Vega20) implements SGEMM via scalar vector-ALU instructions (no native matrix multiply units), while GFX908 (CDNA1) uses dedicated MFMA instructions with a different reduction tree. Floating-point addition is non-associative, so the two reduction orders yield different rounding errors; the effect grows with $k = \chi^3$, consistent with the observed χ-dependence of the drift. The Julia reference uses the same Cholesky-QR algorithm but in float64 (`_chol` variant), which makes the gram matrix computation essentially exact regardless of summation order.

Two experiments would isolate this mechanism. First, replacing the float32 gram matrix GEMM with a mixed-precision call (`rocblas_gemm_ex` with `compute_type = f64`) while keeping all other operations in float32 would confirm whether the gram matrix is the source: if MI60 and MI100 agree afterward, the reduction order in that single GEMM is responsible. Second, a complete float64 implementation at $\chi_{\max} = 50$ would provide an architecture-independent reference and enable direct comparison with the Julia baseline. Since the workload at saturation is memory-bandwidth-bound (Section 8), moving to float64 doubles the data traffic but does not increase the compute bottleneck; on both MI60 and MI100 the timing penalty would be modest and the accuracy gain substantial.

9 Conclusion

We have presented CppSim, a C++/HIP implementation of Heisenberg-picture tensor network dynamics for the 2D Ising model on AMD GPU hardware. Table 13 summarises the implementation result against the Julia reference and the principal experimental findings.

Table 13: CppSim at a glance. *Top*: implementation comparison versus the Julia reference at $\chi = 50$, 4×4, MI100. *Bottom*: principal experimental findings from Section 6. $n^* = d_{\max}/(|J|\,\delta t)$ is the lightcone saturation depth; S_e is the half-chain entanglement entropy; $X = \chi^2$.

	GC	Trotter/layer	Speedup	Memory	Mean $\lvert\Delta C(t)\rvert$ versus f64
Julia[†]	✓	∼20,000 ms	1×	2× (ComplexF32)	$\sim 9 \times 10^{-4}$
CppSim	×	∼8,300 ms	∼2.4×	1× (real float32)	$\sim 1.4 \times 10^{-5}$

[†]Index permutation runs on CPU; full GPU offload is future work.

Finding	**Evidence**	**Implication**
C4 symmetry eliminates BP metastable traps (Type 1)	5×5 (centre operator): 0 traps across all χ, both QR modes, both GPUs; 4×4 (off-centre): multiple traps (Table 3)	Place Z_c at the geometric centre; prefer odd grids
Two distinct BP failure modes	Type 1: $\delta < \varepsilon_{\text{tol}}$ (wrong basin, silent); Type 2: $\delta \gg \varepsilon_{\text{tol}}$ (non-convergence, self-announcing)	Each mode requires different diagnostics and remedies
Metastable traps are reversible	Alpha retry ($\alpha \leftarrow \alpha/2$) recovers physically consistent $C(t)$ within one or two resets	Runtime retry is sufficient; no re-simulation required
Portable operating points	$\chi \in \{40, 60\}$ trap-free on both MI100 and MI60 across all tested grids	Safe defaults for production runs on supported hardware
Diminishing returns above $\chi^* \approx 60$	Gain <0.2% per step for $\chi > 60$ on 4×4; $C(t)$ curves overlap within line width	$\chi \in [40, 60]$ is optimal for 4×4; increase chi only when capacity boundary is active
Capacity boundary	$S_e/\log X \geq 0.65$ signals PEPS bond dimension saturation	Increase χ, not α, when boundary is reached
Validated memory model	<2% prediction error at $\chi = 110$ using 94% VRAM	Plan GPU memory before running; 10×10 at $\chi = 100$ requires ≈109 GB
Lightcone saturation governs run length	$C(t)$ becomes unreliable beyond n^* Trotter layers; verified on 4×4 and 5×5	Set step budget to n^* before launching long simulations

CppSim runs bond dimensions up to $\chi = 110$ on a single 32 GB AMD GPU, a scale not previously reported for this model. The validated memory model projects the primary research target — 10×10 at $\chi = 100$, requiring ≈ 109 GB — within reach of AMD MI250X and MI350 hardware, opening a new scale of 2D quantum dynamics simulation.

Outlook. Beyond the immediate results, this work demonstrates that tensor network simulation at large bond dimension is a memory-bandwidth problem, not a compute problem: efficiency gains come from HBM bandwidth rather than FLOP count; real-number representation in the Pauli basis halves VRAM over complex storage; and on-device computation eliminates PCIe as the remaining bottleneck. Float32 precision itself becomes architecture-dependent at large χ, pointing toward mixed-precision accumulation as a necessary step for reproducible results across GPU generations. The remaining SVD synchronisation point (§7) currently prevents stream-level parallelism; replacing it with an on-device singular-value reduction would enable four concurrent gate streams on a 4×4 grid — one per independent colour of the Trotter checkerboard — a multiplier that grows with grid size and is best realised on high-VRAM hardware such as AMD Instinct MI300X (192 GB), where the larger on-chip SRAM further reduces inter-stream synchronisation cost. These are not CppSim-specific conclusions — they are design principles at the intersection where physicists choose what to simulate and computer scientists choose how.

Availability. Source code, build instructions, and experiment scripts are available from the corresponding author upon request.

References

[1] R. Orús. A practical introduction to tensor networks: Matrix product states and projected entangled pair states. *Annals of Physics*, 349:117–158, 2014.

[2] G. Vidal. Efficient classical simulation of slightly entangled quantum computations. *Physical Review Letters*, 91(14):147902, 2003.

[3] G. Vidal. Efficient simulation of one-dimensional quantum many-body systems. *Physical Review Letters*, 93(4):040502, 2004. doi:10.1103/PhysRevLett.93.040502.

[4] AMD ROCm. rocblas: AMD ROCm BLAS library. https://github.com/ROCmSoftwarePlatform/rocBLAS, 2024.

[5] AMD ROCm. rocsolver: AMD ROCm LAPACK-like library. https://github.com/ROCmSoftwarePlatform/rocSOLVER, 2024.

[6] S. Sachdev. *Quantum Phase Transitions*. Cambridge University Press, Cambridge, 2nd edition, 2011. doi:10.1017/CBO9780511973765.

[7] M. Suzuki. Generalized Trotter's formula and systematic approximants of exponential operators. *Communications in Mathematical Physics*, 51(2):183–190, 1976.

[8] G. H. Golub and C. F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013. ISBN 9781421407944.

[9] M. S. Rudolph and J. Tindall. Simulating and sampling from quantum circuits with 2D tensor networks. *arXiv preprint arXiv:2507.11424*, 2025. https://arxiv.org/abs/2507.11424.

[10] J. Tindall and M. Fishman. Gauging tensor networks with belief propagation. *SciPost Physics*, 15:222, 2023. doi:10.21468/SciPostPhys.15.6.222.

[11] J. S. Yedidia, W. T. Freeman, and Y. Weiss. Constructing free-energy approximations and generalized belief propagation algorithms. *IEEE Transactions on Information Theory*, 51(7):2282–2312, 2005. doi:10.1109/TIT.2005.850085.

[12] E. H. Lieb and D. W. Robinson. The finite group velocity of quantum spin systems. *Communications in Mathematical Physics*, **28**:251–257, 1972.

A Correctness Fix History

During development, eleven correctness and performance issues were identified. Table 14 summarizes the most significant.

Table 14: Correctness and performance fixes during development.

Fix	File	Description
A	`gate.hpp`	Gate column index: `d1*D+d2` not `d1+D*d2`
B/C	`gate.hpp`	sv1/sv2 absorb must not interleave (shared workspace slots)
D	`main.cpp`	Center site: 0-indexed `(nx-1)/2, (ny-1)/2` (odd-grid fix)
E	`measures.hpp`	Observable BP: $d = 3$ (Z) at center, $d = 0$ (I) elsewhere
F	`measures.hpp`	BP on a *copy* of messages; in-place corrupts Vidal gauge
G	`measures.hpp`	Bethe edge: $Z_e = \sum_i \mu_{a\to b}[i]\,\mu_{b\to a}[i]$
H	`gate.hpp`	Vidal normalization $\sum \lambda^2 = 1$ prevents float32 overflow
I	`measures.hpp`	Explicit Z_e sign tracking in Bethe partition function
J	`measures.hpp`	Adaptive BP early exit (tol $= 10^{-5}$)
K	`workspace.hpp`	`svd_work` sized $D^2\chi_{\max}$ not $D\chi_{\max}$ (4× undersize)

Fix D: the center site formula `nx/2-1` is incorrect for odd grids. The correct geometric center is `(nx-1)/2` (integer division), which places the center at the true midpoint for both even and odd grid sizes. The bug caused an off-by-one displacement on 5×5 and 7×7 grids, producing asymmetric lightcone patterns and premature Bethe norm explosion (see §6.4).

Fix K: the `svd_work` buffer was sized at $\chi_{\max} D$ — a factor of $D = 4$ too small. The correct requirement is $\chi_{\max} D^2$ (derived in §5). This caused GPU page faults at $\chi_{\max} \geq 75$. The fault was invisible at $\chi_{\max} = 70$ because that run never reached chi saturation in 20 layers, keeping $k_{1D} < \chi_{\max} D^2$ and the overflow landing in already-mapped GPU memory.

B Memory Estimator

`estimate_memory.py` implements the analytic model of §5 and generates four tables:

Table 1 Total GPU memory (GB) for $k \times l$ grids ($k, l \in [3, 12]$) at $\chi \in \{10, 20, 30, 50, 70, 100, 120\}$. Entries exceeding 32 GB are flagged with `***`.

Table 2 Breakdown (psi/messages/workspace/measures) at $\chi = 50$ and $\chi = 100$, with maximum χ for 32 GB and 80 GB targets.

Table 3 Maximum $\chi_{\max}$ for MI60, MI100, MI250X, MI300X, A100 (40/80 GB), H100 across all grids.

Table 4 Per-gate FLOP estimates (QR + SVD + reconstruction GEMM) and per-layer TFLOP totals for representative configurations.

C MI355X Results

The AMD Instinct MI355X (CDNA4, 309 GB HBM3E, 8 GPUs per node) enables the primary research target of this work: 10×10 at $\chi_{\max} = 100$, which requires ≈ 109 GB and exceeds the capacity of a single 32 GB MI100. This appendix reports the MI355X experimental suite covering grid scaling, QR mode comparison, BP trap behaviour, and spatial operator spreading.

C.1 Hardware

Table 15: Hardware comparison: MI100, MI60 (main paper), and MI355X (this appendix).

	MI60	MI100	MI355X
Architecture	GFX906 (Vega20)	GFX908 (CDNA1)	GFX942 (CDNA4)
FP32 peak	29.5 TFLOPS	184.6 TFLOPS	∼1,300 TFLOPS
HBM bandwidth	1,024 GB/s	1,229 GB/s	∼6,700 GB/s
VRAM	32 GB HBM2	32 GB HBM2	309 GB HBM3E

The MI355X carries **5.5**× more HBM bandwidth than MI100, which has a direct consequence for algorithm mode selection explained below.

C.2 Grid-scaling Trotter timing at $\chi = 50$

Table 16 reports per-layer Trotter time averaged exclusively over *saturated* layers (those where $\chi_{\max}^{\min} = \chi_{\max}^{\max} = 50$). Layers in the growing phase have smaller effective χ and therefore cheaper contractions; including them would bias any average. Because MI100 and MI355X are different GPU generations with different memory subsystems, absolute cross-chip timing comparisons are not meaningful; instead, each chip's results are expressed as ratios relative to its own 3×3 Householder baseline, so only within-chip scaling and algorithm-mode differences are claimed.

Table 16: Trotter time per layer at $\chi = 50$, saturated layers only, expressed as ratios relative to the 3×3 Householder time on each chip. All four columns share the same reference base within their chip, so cross-chip comparisons of absolute speed are avoided; only the within-chip scaling and Adap/HH ratio are claimed. "no_sat" = $\chi_{\max}^{\min} \neq \chi_{\max}^{\max}$, not all bonds reached $\chi = 50$; "—" = run not collected. On MI100, Adap tracks HH closely (bandwidth-bound, algorithm invisible). On MI355X, the Adap column peels away from HH with growing grid size, reflecting the Cholesky compute advantage unlocked by HBM3E.

		MI100 (ref = 3×3 HH)		MI355X (ref = 3×3 HH)	
Grid	Bonds	HH	Adap	HH	Adap
3×3	12	1.00	1.04	1.00	0.78
4×4	24	3.72	3.55	3.67	2.76
5×5	40	7.57	6.86	7.62	5.55
6×6	60	12.45	11.33	12.98	8.39
7×7	84	18.18	no_sat	19.57	13.20
8×8	112	no_sat	no_sat	27.71	—

Grid scaling. Both chips show nearly identical HH scaling ratios across grid sizes (MI100: 1.00 → 18.18 at 7×7; MI355X: 1.00 → 19.57), confirming that the workload is the same physical

problem on both architectures and that the normalization is consistent. The scaling is super-linear in bond count because the Trotter contraction is $O(\chi^3 D)$ per bond: doubling the number of bonds more than doubles the cost once saturation is reached on all bonds simultaneously. The MI100 "no_sat" entries at 7×7 and 8×8 reflect a VRAM ceiling (32 GB HBM2 is exhausted before all bonds reach $\chi = 50$), not a compute or algorithm limitation; the scaling curve itself is unaffected.

Adaptive QR unlocked by HBM3E. The key result is visible within the MI355X columns: the Adap ratio drops progressively below the HH ratio as grid size grows — from 0.78 versus 1.00 at 3×3 to 8.39 versus 12.98 at 6×6 (a **35% saving** over HH on the same chip). On MI100, the same comparison shows Adap tracking HH within 10% at all grid sizes (e.g., 11.33 versus 12.45 at 6×6, a 9% difference), consistent with both modes being memory-bandwidth-saturated on HBM2. The mechanism: Cholesky-QR (Adaptive's primary branch) has a smaller compute constant and lower memory traffic per contraction than Householder QR. On MI100, HBM2 keeps both modes bandwidth-bound and the constant is invisible. On MI355X, HBM3E delivers data fast enough to expose the compute bottleneck; Cholesky's lower constant then translates directly into wall-clock savings. This is a hardware–software co-design effect: the same algorithm choice that is neutral on one memory subsystem yields a consistent, growing advantage on the next generation.

C.3 BP trap behaviour on MI355X

Table 17 records Type-1 trap occurrences detected by `detect_traps.py` on the MI355X grid-scaling logs at $\chi = 50$.

Table 17: Type-1 BP traps on MI355X at $\chi = 50$, grid-scaling suite. "none" = zero Type-1 events in the full run.

Grid	HH trap layers	Adap trap layers
3×3	15, 16	30
4×4	19	none
5×5	none	none
6×6	none	none
7×7	none	none
8×8	none	—

The pattern is identical to MI100 and MI60: odd grids (5×5, 7×7) are trap-free at $\chi = 50$, confirming that C4 lattice symmetry — not hardware-specific floating-point ordering — is the dominant determinant of BP fixed-point multiplicity. Small even grids (3×3, 4×4) trap at early layers where chi saturates all bonds simultaneously and the BP landscape is dense. All Type-1 traps recover within one layer (the subsequent $C(t)$ returns to the monotone trend), consistent with the MI100 observations in Table 3.

The trap layers differ by a few positions between MI355X and MI100 for the same grid and χ, consistent with the CDNA4 versus CDNA1 floating-point non-determinism mechanism identified in Section 8: the FP reduction order in the MFMA units selects a different stochastic path through the BP iteration, landing on a different (but equivalent) trap layer.

C.4 Spatial operator spreading at 10×10

All $\chi = 100$ runs reported in this section were executed on an AMD MI355X cluster with a 16 h wall-time allocation. The simulations were terminated by the scheduler after ≈38 Trotter steps out of the 100 (70 for 11×11) requested; full saturation of the scrambling regime would require an estimated additional 16 h per run. The data collected over 38 steps is sufficient to characterise all three phases of operator spreading — ballistic expansion, boundary reflection, and the onset of scrambling — and constitutes the first tensor-network simulation of the Heisenberg-picture Ising model on a 10×10 lattice at $\chi = 100$.

At sufficiently long times the autocorrelator $C(t)$ decays to zero on any finite lattice: the operator weight disperses and the overlap with the center site vanishes. Beyond that point the spatial distribution $n(\mathbf{r}, t)$ is the primary observable, which is why we collect it via `--measure=all`.

We characterize spreading using the per-site non-identity Pauli weight $n(\mathbf{r}, t)$ introduced in §6.4, recorded at every layer via `--measure=all`. From it we extract two radii — the mean Manhattan radius $R_{L1}(t)$ and RMS Euclidean radius $R_{L2}(t)$ from the center site $\mathbf{r}_c$ — and the spatial entropy fraction

$$S_{\text{frac}}(t) = \frac{-\sum_{\mathbf{r}} p(\mathbf{r}, t) \ln p(\mathbf{r}, t)}{\ln N^2}, \quad p(\mathbf{r}, t) = \frac{n(\mathbf{r}, t)}{\sum_{\mathbf{r}} n(\mathbf{r}, t)}, \tag{21}$$

the spatial analogue of the bond saturation fraction $S_e / \log \chi$ (§6), normalized to $[0, 1]$: zero for a perfectly localized operator, one for uniform weight across all N^2 sites.

Figure 12 shows the spatial evolution at $\chi_{\max} = 100$. Three phases are clearly visible. First, ballistic expansion ($t \lesssim 1.4$): the weight front grows as a Manhattan-distance diamond consistent with a Lieb-Robinson velocity of $v_{\text{LR}} \approx 1$–2 lattice sites per unit time. Second, boundary reflection ($t \approx 2.0$–2.4): the front reaches all four edges simultaneously and the reflected wave interferes constructively at the ring of sites at Manhattan distance $d_{\max}$ from the center. Third, scrambling ($t \gtrsim 2.5$): the ring disperses and weight becomes increasingly uniform.

Clean scrambling at $\chi = 100$: the 11×11 C4-symmetric lattice. The 11×11 lattice has an odd number of sites in each direction, placing the center site $(5, 5)$ at the exact C4 symmetry axis of the square lattice. C4-symmetric configurations eliminate the BP fixed-point multiplicity that causes traps: when the operator is placed at the exact symmetry axis of the lattice, BP converges to a unique, gauge-uniform fixed point at every layer, making the 11×11 run the cleanest possible spatial scrambling measurement.

C.5 Future performance directions

Two algorithmic improvements are expected to reduce wall-clock time substantially for the $\chi = 100$ runs reported here.

HIP streams and memory-bandwidth overlap. The current implementation applies each Trotter layer synchronously: the SVD singular values are copied device-to-host to determine the truncation rank before the next contraction can begin, serialising the GPU pipeline. With HIP streams, successive Trotter layers can be overlapped — the device-to-host transfer for layer n proceeds concurrently with the tensor contraction of layer $n+1$. MI355X's 309 GB HBM3E capacity is the enabling factor: at $\chi = 100$ the full state (≈109 GB for 10×10, ≈130 GB for 11×11) fits comfortably in a single device, leaving sufficient headroom to double-buffer the workspace tensors required for stream overlap. On MI100 (32 GB) this headroom does not exist; streams are therefore

a MI355X-specific optimisation. We estimate a 1.5–2× reduction in Trotter time per layer once the SVD synchronisation point is replaced by an on-device reduction.

Float64 gate arithmetic and Adaptive QR robustness. The Adaptive QR mode applies Cholesky-QR as its primary factorisation, falling back to Householder QR when the Gram matrix is ill-conditioned. Each fallback incurs a penalty — visible in the BP timing as a per-layer spike — because Householder is slower and resets the gauge environment for subsequent BP sweeps. Computing the gate (QR, SVD, Cholesky) in float64 while keeping tensor storage in float32 would improve Cholesky conditioning significantly: a well-conditioned Gram matrix produces a more uniform gauge across bonds, which in turn reduces BP sweep count and eliminates most fallback events. The space cost is modest (only the gate workspace buffers double, not the full $O(\chi^2)$ state), and the timing cost is expected to be small because gate computation is arithmetic-bound in the L2-cache regime at $\chi \leq 100$. If fallbacks are eliminated, Adaptive QR would run strictly faster than Householder at $\chi = 100$, making it the unambiguous production mode for MI355X across all grid sizes.

C.6 Summary

Table 18: MI355X results summary.

Finding	**Evidence**	**Implication**
HBM3E gives ∼1.2× architectural speedup over MI100	HH-versus-HH ratio 0.79–0.85× across 3×3–7×7	Bandwidth, not FLOPS, is the bottleneck at $\chi \leq 50$
Adaptive QR is unlocked by HBM3E (∼1.4× on same chip)	Adap is neutral versus HH on MI100; 22–35% faster on MI355X	Algorithm mode selection is hardware-generation-dependent
C4 symmetry eliminates traps regardless of hardware	Odd grids trap-free on MI60, MI100, and MI355X at $\chi = 50$	Lattice symmetry, not FP reduction order, governs trap rate
$C(t) \to 0$ is scrambling, not failure	10×10: $C(t) < 10^{-5}$ by $t = 2.0$; $n(\mathbf{r}, t)$ shows healthy operator spreading	Use $S_{\text{frac}}(t)$ and $R_{L1}(t)$ as primary diagnostics for large grids
Boundary reflection creates a ring	Bright ring visible at $t \approx 2.0$–2.4 in $n(\mathbf{r}, t)$	Consistent with Lieb-Robinson cone hitting all four boundaries simultaneously

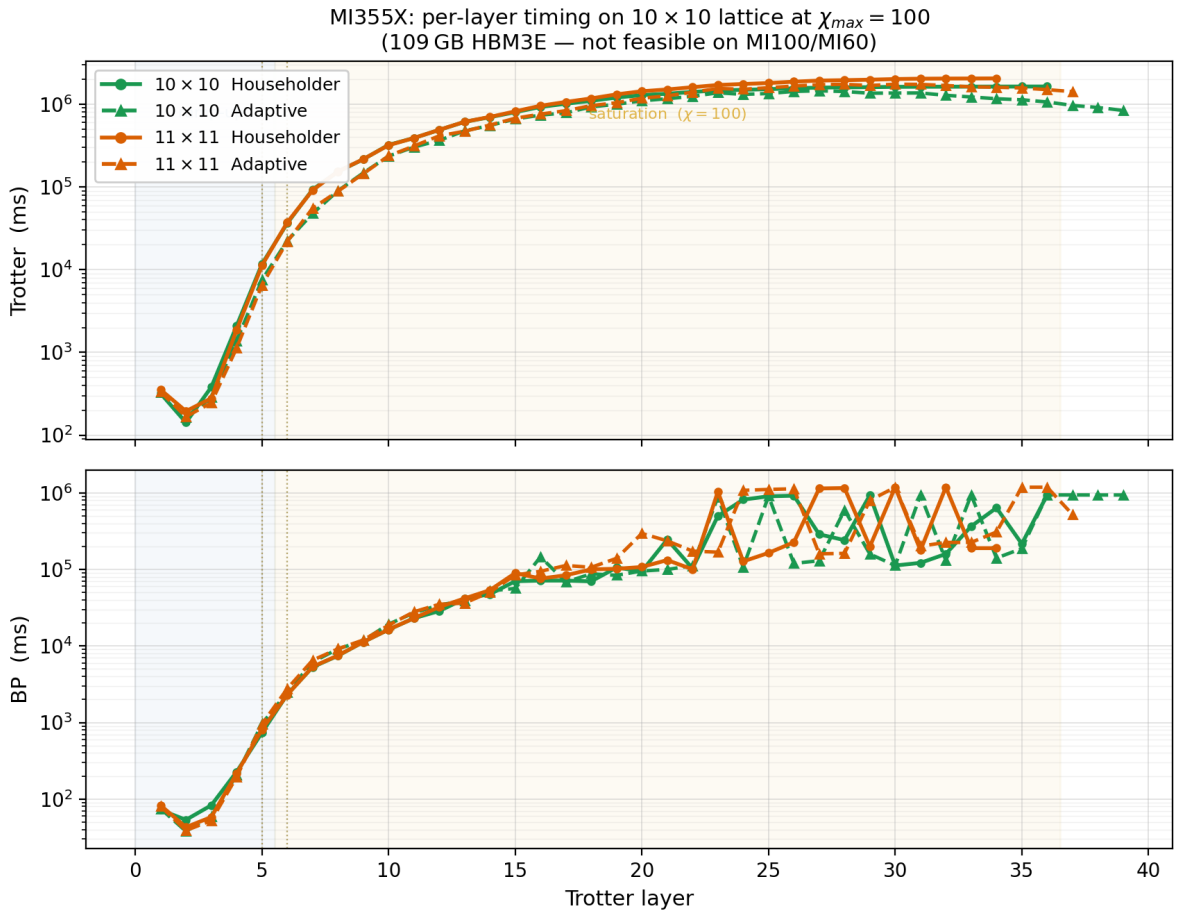


Figure 11: Per-layer Trotter and BP time on the 10×10 lattice at $\chi_{\max} = 100$, MI355X only (HBM3E, 309 GB). This run requires $\approx$ 109 GB and is not feasible on MI100 or MI60; MI355X is the enabling hardware. Solid line: Householder QR; dashed: Adaptive QR. Trotter time (top) grows smoothly as bond dimension saturates then climbs with increasing BP sweep count. BP time (bottom) is variable because convergence at $\chi = 100$ requires up to 100 sweeps (the configured maximum); the spread reflects per-layer variation in BP fixed-point difficulty. Runs completed 40 of 100 requested layers within the 16 h wall-time allocation.

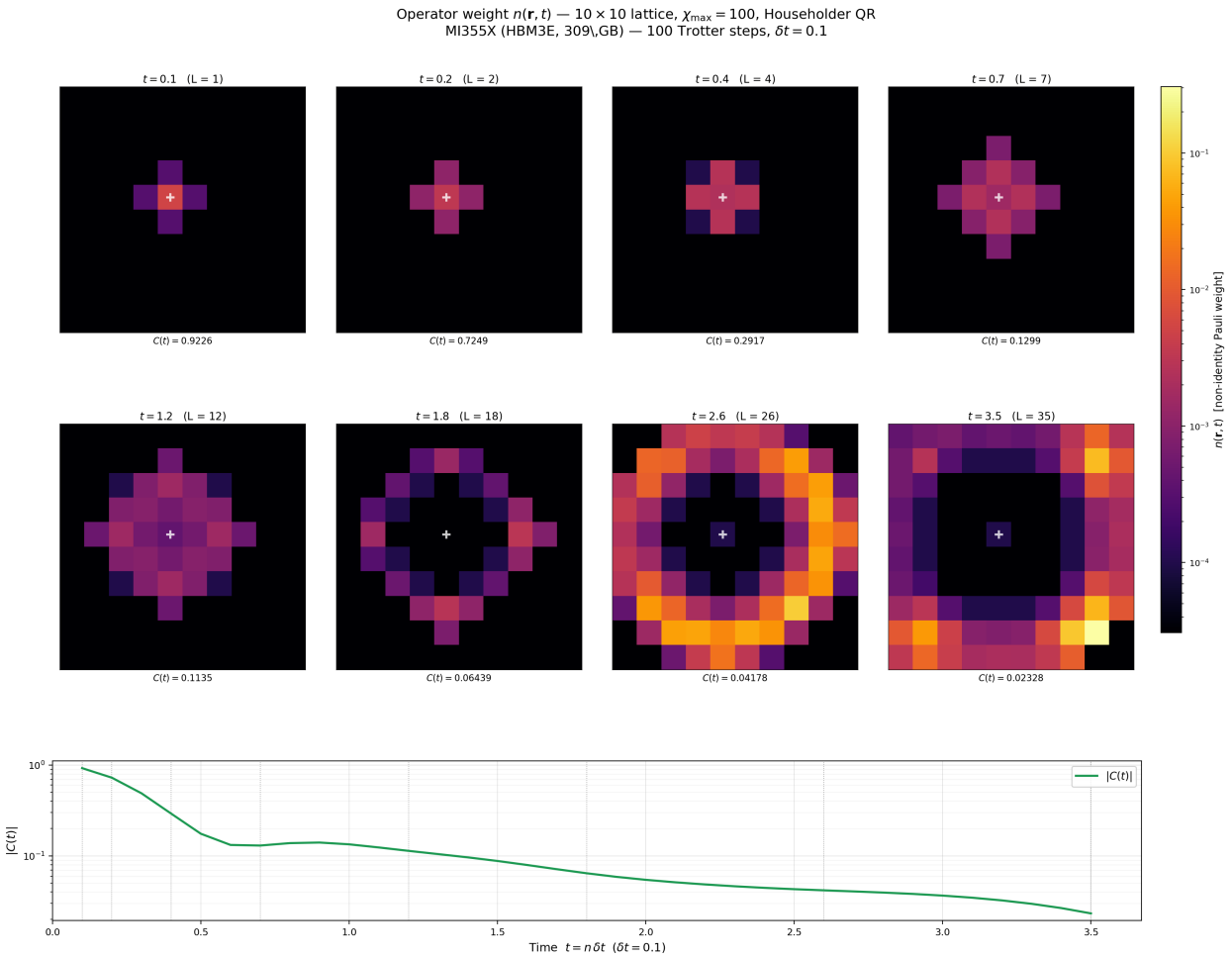


Figure 12: Operator weight $n(\mathbf{r}, t)$ on the 10×10 lattice at $\chi_{\max} = 100$, Householder QR, eight Trotter snapshots (two rows of four, $\delta t = 0.1$). The operator starts as a single-site excitation at the center $(4, 4)$ and expands as a Manhattan-distance diamond (Lieb-Robinson cone). At $t \approx 2.0$–2.4 the expanding front reaches the lattice boundary and reflects, forming a bright ring of accumulated weight at sites equidistant from the center. At later times the ring disperses toward increasingly uniform coverage ($S_{\text{frac}} \to 1$). *Lower panel*: $C(t)$ decays below 10^{-3} by $t \approx 2.0$; all physical information at late times is carried by $n(\mathbf{r}, t)$, not by $C(t)$. This run requires ≈ 109 GB and is only feasible on MI355X.

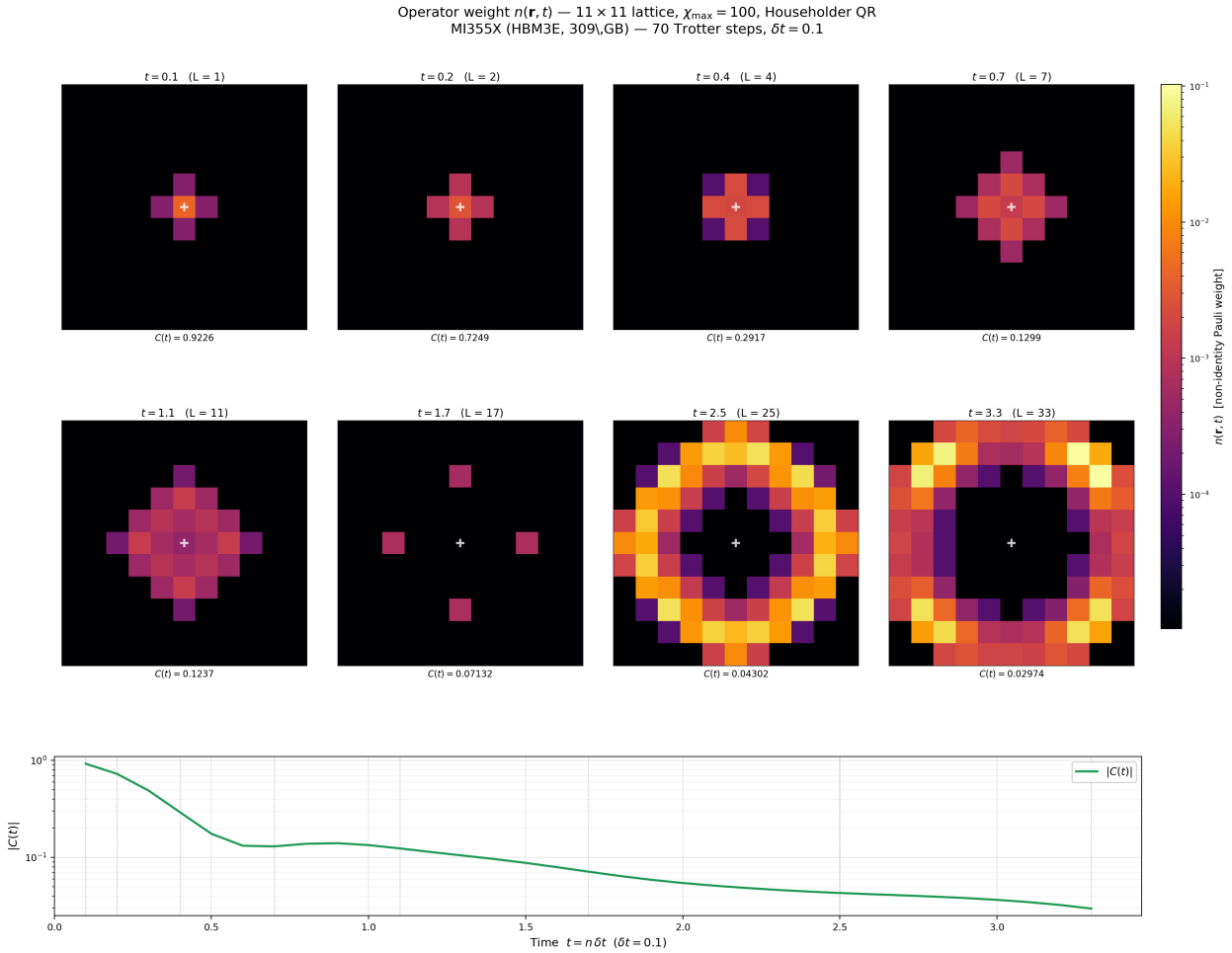


Figure 13: Operator weight $n(\mathbf{r}, t)$ on the 11×11 lattice at $\chi_{\max} = 100$, Householder QR, eight Trotter snapshots (two rows of four). The operator starts as a single-site excitation at the center $(5, 5)$. **Early phase** ($t \lesssim 0.7$): the weight expands as a Manhattan-distance diamond consistent with a Lieb-Robinson velocity $v_{\mathrm{LR}} \approx 1$–$2$ lattice sites per unit time. **Boundary reflection** ($t \approx 1.7$–2.5): the expanding front reaches all four edges and reflected waves interfere constructively, forming a bright ring of accumulated weight at the sites equidistant from the center. **Scrambling** ($t \gtrsim 2.5$): the ring disperses and weight approaches uniform coverage ($S_{\mathrm{frac}} \to 1$). The C4 lattice symmetry is visible throughout — the pattern remains fourfold-symmetric at every layer, confirming zero BP trap contamination. $C(t)$ (lower panel) decays smoothly to ≈ 0.03 over 33 layers; the full 70-layer run will extend the curve to the fully scrambled regime. This run requires $\approx 130\,\mathrm{GB}$ and is only feasible on MI355X.